Profiling Android Instrumentation-Test Execution Styles in CI: Performance, Predictability, and Verdict Usability

***Abstract*—Android instrumentation tests provide high-fidelity end-to-end validation but are operationally difficult to manage in continuous integration (CI) because feedback time is shaped not only by test execution, but also by setup/provisioning, orchestration overhead, environmental instability, and heavy-tailed runtimes. Prior work identified recurring execution styles for running instrumentation tests in CI (Community actions, Custom setups, Gradle Managed Devices (GMD), and Third-Party device farms), yet practitioners still lack a principled way to compare these styles under different operational objectives. In this paper, we present a telemetry-driven evaluation method for operational profiling of CI-based instrumentation testing, instantiated and empirically validated on Android instrumentation testing in GitHub Actions. The method combines workflow evidence with run/job/step telemetry to construct a study-facing run×style dataset, define two aligned timing layers, separate comparable execution regimes, and compare styles along four dimensions: speed, predictability, overhead composition, and deployment/outcome context. To reduce confounding in observational comparisons, we analyze timing-focused results under a controlled regime of instrumentation-executed first-attempt runs with usable verdicts, and use a workflow-shape fingerprint encoded as a signature hash as a downstream robustness layer. On a dataset of 8,910 executed run×style records from GitHub projects, the method reveals distinct operational profiles: Community provides the fastest typical completion but higher variability and a distributed overhead profile; GMD provides the strongest predictability-first profile, the clearest fast-entry behavior, and an execution-centric overhead structure; Third-Party combines slow sustained execution with the strongest absolute tail-risk profile and a heavy-entry plus heavy-execution pattern; and Custom remains a sparse but distinct mixed case with competitive entry and a much heavier completion tail. The deployment/outcome analysis further shows that styles differ in usable-verdict rate, success rate among usable outcomes, and trigger context. We synthesize these findings into an operational performance profile and a profile-derived decision-support guide that maps CI objectives to style choices and first optimization targets.**

***Index Terms*—Android instrumentation testing, continuous integration, CI telemetry, GitHub Actions, operational profiling, telemetry-driven evaluation, feedback speed, predictability, overhead composition, usable verdicts, deployment context, workflow-shape robustness, workflow mining, emulator testing.**

I. Introduction

Android instrumentation testing is indispensable for validating mobile behavior under realistic execution conditions, yet it is operationally difficult to manage in continuous integration (CI) because feedback time is shaped not only by test execution, but also by emulator setup, provisioning, orchestration, artifact handling, and environmental instability. Prior work identified recurring instrumentation-testing execution styles in CI—Community actions, Custom setups, Gradle Managed Devices (GMD), and Third-Party device farms—and showed that these styles are widely used in GitHub-based Android projects [1]. However, practitioners still lack a principled way to compare these styles under different operational objectives.

A central challenge is that CI comparison is easily misleading when it relies only on end-to-end runtime. Two styles may show similar completion times while differing substantially in predictability, tail risk, overhead placement, or outcome usability. Likewise, observational comparisons are confounded by attempt regime, trigger context, and workflow structure. In practice, developers and release owners need more than a single duration metric: they need an evaluation basis that explains how quickly feedback is delivered, how stable that feedback is, where latency accumulates, and whether runs produce usable outcomes in the deployment context where they are actually used.

In this paper, we therefore frame the problem as a *telemetry-driven evaluation method* for operational profiling of CI-based instrumentation testing. The method is instantiated and empirically validated on Android instrumentation testing in GitHub projects. It uses workflow evidence together with GitHub Actions run/job/step telemetry to construct a study-facing run×style dataset, define aligned timing layers, isolate comparable execution regimes, and derive style-level operational profiles from historical CI executions. This framing is motivated by the increasing availability of CI telemetry and by prior evidence that CI performance and failures are strongly shaped by build duration, reruns, infrastructure variation, and workflow structure [2]–[6].

The proposed evaluation method uses three core operational dimensions as its main comparison basis. First, *speed* captures how quickly a style delivers feedback. Second, *predictability* captures stability and tail exposure, which matter because a style that is fast in typical cases may still be operationally costly if it exhibits long or unstable tails. Third, *overhead composition* explains where latency accumulates, enabling style differences to be linked to practical optimization levers rather than treated as opaque runtime gaps. These three dimensions provide a coherent comparison basis because they answer complementary operational questions: how fast feedback is, how dependable it is, and where intervention should be directed. We then extend this basis with a fourth, practice-facing dimension: *deployment and outcome context*, which captures how styles

are actually used across CI triggers and how often they yield usable and successful run-level outcomes.

Methodologically, the paper makes four main contributions. First, it introduces a telemetry-driven evaluation design that constructs a study-facing run×style dataset from workflow, run, job, and step telemetry and organizes timing measurement through two aligned layers: a broad job-level decomposition and a more precise step-level decomposition. Second, it defines a regime-aware comparison strategy that separates timing-focused analyses from deployment/outcome analyses and validates the main timing regime using a three-step validation plan. Third, it introduces a workflow-shape fingerprint encoded as a signature hash and uses it as a downstream robustness layer for checking whether the main conclusions remain visible across comparable workflow settings. Fourth, it demonstrates the method on a large snapshot of Android instrumentation testing in GitHub Actions and synthesizes the results into an interpretable operational profile and a profile-derived decision-support guide.

Empirically, the method reveals that the four styles exhibit distinct operational profiles rather than a single global ranking. Community provides the fastest typical completion but with greater variability and a more distributed overhead profile. GMD provides the strongest predictability-first profile, the clearest fast-entry behavior, and an execution-centric overhead structure. Third-Party combines slow sustained execution with the strongest absolute tail-risk profile and a heavy-entry plus heavy-execution pattern. Custom remains a sparse but distinct mixed case, with competitive entry but a much heavier completion tail. The deployment/outcome analysis further shows that styles differ not only in timing, but also in usable-verdict rate, success rate among usable outcomes, and trigger context. Together, these results support the paper's central claim: Android instrumentation-testing styles in CI are best understood through a telemetry-driven operational profile rather than through runtime alone.

The remainder of the paper is organized as follows. Section II reviews the necessary background and related work. Section III presents the proposed telemetry-driven evaluation method, including dataset construction, timing decomposition, comparison regime, validation, and research-question specifications. Section IV reports the empirical validation of the method through the Android instrumentation-testing snapshot, covering speed, predictability, overhead composition, and deployment/outcome context. Section V synthesizes these results into an operational performance profile and a profile-derived decision-support guide. Section VI discusses implications, Section VII reviews threats to validity, and Section VIII concludes.

II. Background and Related Work

A. Mobile Instrumentation Testing in Continuous Integration

Android instrumentation tests execute end-to-end scenarios on emulators or physical devices and are widely used to validate application behavior under realistic runtime conditions (UI, lifecycle, permissions, networking, and device interactions) [7]–[9]. These tests run "on-device" and therefore inherit environment- and timing-sensitive behavior that is uncommon in local JVM tests. Compared to unit or JVM-level tests, instrumentation testing is operationally challenging in CI due to (i) provisioning and booting devices/emulators, (ii) dependence on SDK/emulator tooling and configuration drift, and (iii) nondeterministic runtime conditions (e.g., UI synchronization, background services, and device variance) that can amplify flakiness and tail latency [10]–[13]. As a result, teams adopt heterogeneous execution approaches and trade off feedback speed, predictability, verdict usability, and maintenance effort.

Our baseline study contributes (i) a candidate-project identification procedure for locating repositories that run instrumentation tests at scale and (ii) an execution-style taxonomy capturing how projects realize instrumentation testing across execution environments and tooling choices [1]. Building on that foundation, the present work focuses on how these styles translate into *developer-feedback operational performance* (speed, predictability, and overhead concentration), and how verdict/usability and trigger deployment refine those operational profiles.

B. Execution-Environment Styles and Mobile CI Substrates

Modern mobile CI pipelines realize instrumentation tests through distinct execution substrates: self-hosted or hosted emulators, build-integrated managed devices, and cloud device farms. Gradle Managed Devices (GMD) formalize a build-managed approach where the Android Gradle Plugin manages device creation/deployment/teardown around instrumented test execution, emphasizing repeatability and execution-centric workflows [**?**]. Cloud device-farm offerings (e.g., Firebase Test Lab) provide remote execution of instrumentation tests on physical and virtual devices and expose orchestration workflows that typically include artifact/result publication after execution [14, 15]. These substrates motivate the style-level comparisons in this paper: they differ not only in typical runtime but also in where overhead concentrates (entry/setup vs. in-window execution vs. post-test completion work) and in how teams deploy them across triggers.

C. CI Telemetry as a Basis for Developer-Feedback Performance

CI platforms expose run-level telemetry (timestamps, outcomes, attempt numbers, and structure) that supports measurement of developer-feedback properties such as verdict completeness and time-to-feedback. In GitHub Actions, workflow-run/job endpoints provide structured access to run/job/step metadata [16, 17], enabling large-scale measurement without relying on repository-local logging conventions. However, telemetry-based studies emphasize that observed outcomes can be shaped by factors beyond code changes, including infrastructure effects, history/queueing, and workflow configuration; these effects complicate attribution if context is not modeled [2, 4, 18, 19]. Accordingly, regime-aware design

and robustness checks are important when drawing cross-style conclusions from observational telemetry.

D. Mining and Characterizing GitHub Actions Workflows at Scale

GitHub Actions has become a major CI substrate for open-source development, motivating datasets and mining work on workflow usage, evolution, and maintenance. Prior studies examine how repositories adopt Actions and how development indicators shift after adoption [5]. Other work characterizes workflow evolution and maintenance, including common modification types and maintenance drivers [20]–[22]. Ecosystem studies also show that workflow reuse concentrates on a small set of popular actions and that many workflows run outdated action versions, creating potential reliability and security risks that can influence operational behavior [6]. These findings motivate our staged pipeline (workflow evidence + run/job/step telemetry) and reinforce the need to treat workflow structure and ecosystem-level drift as part of the operational context.

E. Explaining Performance via Step-Level Telemetry and Overhead Decomposition

End-to-end duration is an intuitive completion-time proxy, but it is often insufficient for diagnosis because multiple pipeline phases contribute to runtime. GitHub Actions exposes job- and step-level telemetry (step start/end timestamps and structure) through workflow-job data [17], enabling decomposition into operationally meaningful phases (setup/provisioning, test execution, provider/artifact handling, and other post-processing). This diagnostic orientation aligns with evidence that developers rely on execution logs and related telemetry for operational understanding and benefit from abstractions that summarize low-level signals into interpretable categories [23]. Accordingly, our analysis treats step-level traces as an "explanation layer" that complements run-level timing and outcomes, supporting bottleneck localization and mapping empirical dominance patterns to plausible first optimization targets.

F. Workflow-Shape Fingerprints and Signature Hashes for Robustness

A recurring challenge in observational performance studies is that outcomes depend not only on the factor of interest (execution style) but also on *execution shape*—workflow structure and complexity (job/step scale, orchestration fan-out, and environment class). Workload characterization surveys emphasize that coarse execution characterizations can group observations into more comparable regimes and reduce the risk that conclusions are dominated by atypical cases [24]. Systems research similarly shows that classifying heterogeneous executions by resource and QoS profiles improves reasoning about performance and variability [25]. In our setting, the analogue is a workflow-shape fingerprint: a compact signature derived from runner family and coarse workflow complexity, used strictly as a robustness layer (influence-trimmed sensitivity checks and within-signature validation). We encode this fingerprint as a stable signature hash; cryptographic hash standards provide a well-established basis for producing consistent identifiers from structured inputs [26].

G. Confounding in Observational CI Studies and the Role of Controls

Because projects select execution styles and workflow structures based on constraints and preferences, comparisons across styles are observational and susceptible to confounding. Methodological work on observational inference emphasizes adjustment for covariates influencing both assignment and outcomes [18, 19]. In CI settings, these concerns are heightened because infrastructure variability and workflow configuration can affect both latency and failure modes [2, 4]. We therefore adopt a regime-aware control strategy: restricting analyses to comparable contexts (verdict-complete executions and first attempts) and complementing pooled results with workflow-shape robustness checks. This aligns with CI evidence that failures and reruns form distinct operational regimes and can distort observed timing distributions [3, 27, 28].

H. Flakiness, Instability, and Mobile-Specific Threats to Predictable Feedback

A key driver of unreliable and unpredictable feedback is test flakiness and environmental instability. Surveys and empirical studies document causes, impacts, and responses to flaky tests and emphasize that flakiness complicates both correctness and performance reasoning [10]. UI and mobile tests are particularly exposed due to larger input spaces and runtime variability; empirical evidence on flaky UI tests (including Android UI tests) highlights recurring root causes and the operational difficulty of stable feedback [11]–[13]. These findings motivate our emphasis on tail behavior and regime-aware comparisons and support our focus on actionable levers that reduce variability and overhead rather than only optimizing medians.

I. From Measurement to Actionable Guidance and Interpretable Rules

Beyond profiling, CI research increasingly aims to produce actionable guidance from telemetry-derived evidence. For operational use, guidance must remain interpretable for practitioners who must trust and act on recommendations. CART decision trees provide classic human-readable rules mapping observable conditions to actions [29]. The broader literature argues for interpretable models in high-stakes or high-cost decision contexts [30], aligning with our profile-derived decision-support structure. Because our corpus is imbalanced across styles and some strata are sparse, we present an interpretable rule list rather than training a potentially majority-class recommender, and position automated recommender learning/validation as future work.

J. Reporting and Validity Considerations for Empirical Software Engineering

Finally, this work follows established guidance for designing, reporting, and interpreting empirical software engineering

Fig. 1. Study-facing data construction pipeline and datasets by stage (snapshot: Mar. 2026). The pipeline is shown after excluding `Real-Devices` from the four-style analytical scope.

studies [31, 32]. These guidelines emphasize transparency about data sources, analysis units, and threats to validity (including confounding), supporting our staged pipeline design and our emphasis on both outcome measurement and robustness-oriented comparisons.

III. Methodology

This study builds on the candidate-project identification procedure and the instrumentation-testing execution-style taxonomy defined in [1]. Following that taxonomy, we focus on the four emulator-based execution styles: **Community**, **Custom**, **GMD**, and **Third-Party** [1]. We exclude `Real-Devices` from the study-facing analytical pipeline because they are sparse in our data and reflect operational assumptions that differ qualitatively from emulator-based CI execution.

A. Dataset construction and extraction pipeline

Our workflow has two parts: an *extraction pipeline* that collects historical GitHub Actions telemetry, and a subsequent *dataset construction* process that transforms those records into the study-facing analytical dataset. The extraction side combines the workflow-screening and style-detection heuristics introduced in [1] with historical run/job/step telemetry from the GitHub Actions REST API [16, 17]. We rely on API-exposed telemetry rather than raw logs because it provides a scalable and structured historical view across repositories, while raw logs and artifacts may be missing or truncated under repository retention policies [33].

The extraction pipeline first identifies relevant workflows and then enumerates their historical executions at the **run level**, which is the level highlighted in Fig. 1. A derived **run×style** view is then created because a single workflow run may expose more than one emulator-testing style. Dataset construction subsequently restricts this inventory to instrumentation-executed records and enriches it with analysis-facing fields, producing the final *MainDataset*.

a) Define inputs and scope.: We begin from **481 candidate repositories** identified by the baseline procedure [1]. The study-facing scope is restricted to the four emulator-based styles (**Community**, **Custom**, **GMD**, **Third-Party**). `Real-Devices` are excluded for consistency across the methodology, figures, and empirical analyses.

b) Data extraction pipeline.: The extraction pipeline consists of three stages.

Stage 1: Workflow inventory and screening. Stage 1 builds a broad workflow inventory by scanning GitHub workflow configuration files, including repository-local references used by those workflows. In the current snapshot, Stage 1 identifies **1,637 workflow records** from **306 repositories**. This stage defines the screened workflow universe rather than the final study-ready set.

Stage 2: Run-level telemetry extraction and normalization. Stage 2 enumerates historical runs for the Stage 1 workflow inventory and extracts the run/job/step telemetry needed for downstream timing analysis. At the study-facing level, this stage is centered on the **run** as the primary extracted execution unit. In the current snapshot, Stage 2 yields **22,338 runs** from **178 run-observed instrumentation-capable workflows** across **140 repositories**. This stage also constructs the broad job-level timing layer later used as Layer 1. Because a single workflow run may involve more than one execution style, Stage 2 additionally materializes a derived **run×style** view. After restricting this derived view to the four study styles and excluding `Real-Devices`, the inventory contains **22,407 run×style records**.

Stage 3: Instrumentation-executed filtering. Stage 3 filters the Stage 2 run×style inventory to records where instrumentation testing actually executed, using style-specific instrumentation-job evidence. After restricting to the four study styles, Stage 3 yields **8,910 instrumentation-executed run×style records**. These records form the extraction output that feeds dataset construction.

c) Dataset construction.: The extracted instrumentation-executed run×style inventory is then transformed into the final study-facing dataset and its analysis layers.

MainDataset construction. The final MainDataset is defined at the **run×style** level and is constructed from the Stage 3 instrumentation-executed inventory. It combines: (i) run-level metadata and broad Layer 1 timing components, (ii) step-derived Layer 2 timing components, and (iii) analysis-facing fields used in the comparative, overhead, and robustness analyses. In the current snapshot, the MainDataset contains **8,910 run×style records**.

Controller subset (Base). The study uses one main controlled timing-comparison regime, denoted **Base**. Base keeps only instrumentation-executed records that are **first attempts** and that have a **usable verdict**, where usable verdict means a terminal run conclusion of `success` or `failure`. Applying these two controllers yields **7,803 run×style records**.

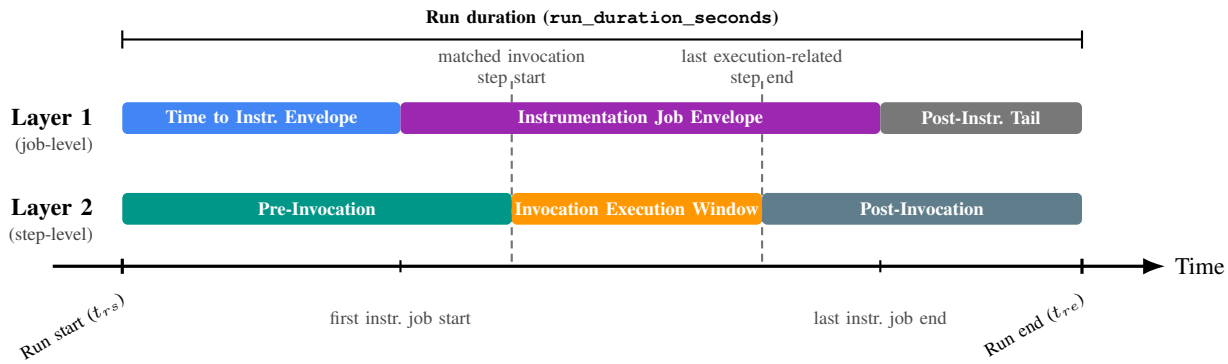

Fig. 2. Study-wide run-timeline decomposition. Layer 1 provides the broad job-level view, and Layer 2 provides the more precise step-level view. Both layers span the same run duration.

Workflow-shape fingerprint layer. The workflow-shape fingerprint layer is derived after MainDataset construction by grouping run×style records with a workflow-shape signature so that structurally similar runs can be compared in follow-up robustness checks. In the current snapshot, fingerprint coverage is complete for the four-style MainDataset.

B. Timeline model and cut-point operationalization

Figure 2 defines the study-wide timing model. The methodology uses two aligned timing layers over the same run duration: a broad job-level layer (**Layer 1**) and a more precise step-level layer (**Layer 2**). Layer 1 provides the durable high-level comparison basis across styles, while Layer 2 refines the same run duration using step telemetry when such telemetry is available.

a) Global run cut points.: We define **run start** (t_{rs}) and **run end** (t_{re}) from workflow-run telemetry. These cut points define **RunDuration** as:

$$\text{RunDuration} = t_{re} - t_{rs}.$$

b) Layer 1: broad job-level decomposition.: Layer 1 is derived from run/job telemetry and is defined by four cut points: run start, first instrumentation-related job start, last instrumentation-related job end, and run end. Using these cut points, we define:

- **Time to Instrumentation Envelope**: run start → first instrumentation-related job start,
- **Instrumentation Job Envelope**: first instrumentation-related job start → last instrumentation-related job end,
- **Post-Instrumentation Tail**: last instrumentation-related job end → run end.

These three components sum to **RunDuration**.

c) Layer 2: precise step-level decomposition.: Layer 2 is derived from Stage 3 step telemetry and is defined by four cut points: run start, matched invocation step start, last execution-related step end, and run end. Using these cut points, we define:

- **Pre-Invocation**: run start → matched invocation step start,
- **Invocation Execution Window**: matched invocation step start → last execution-related step end,
- **Post-Invocation**: last execution-related step end → run end.

These three components also sum to **RunDuration**. This definition centers the middle component on the observable invocation path and avoids absorbing later non-execution continuation into the core execution window.

d) Complementary timing layers.: Layer 1 and Layer 2 are complementary observational views of the same run duration. Layer 1 provides the broad job-level decomposition, while Layer 2 provides the more precise invocation-centered step-level decomposition when step telemetry is available.

C. Study variables and roles (at-a-glance)

Figure 3 provides a compact overview of the variables used in the study. These variables play four roles. First, the timing outcomes capture performance using the two timeline layers introduced earlier. Second, the Base controls define the main comparison subset used in the core timing analyses. Third, the grouping dimensions support the interpretation of where overhead occurs and what kind of work dominates it. Fourth, the workflow-shape fingerprint variables are used only as a downstream robustness layer checked in Validation Step 3 and used in later follow-up analyses, rather than as primary comparison factors.

In the current study design, Base is defined by **first-attempt usable-verdict** records for timing-focused analyses. The practice-facing outcome analysis in RQ4 uses instrumentation-executed first attempts without the usable-verdict restriction.

D. Measures and data quality

a) Main comparison factor.: The study compares the four emulator-based execution styles: **Community**, **Custom**, **GMD**, and **Third-Party**.

b) Controllers.: To improve comparability, the main timing analyses focus on a controlled subset that keeps only instrumentation-executed runs, first attempts, and usable verdicts (`success` or `failure`).

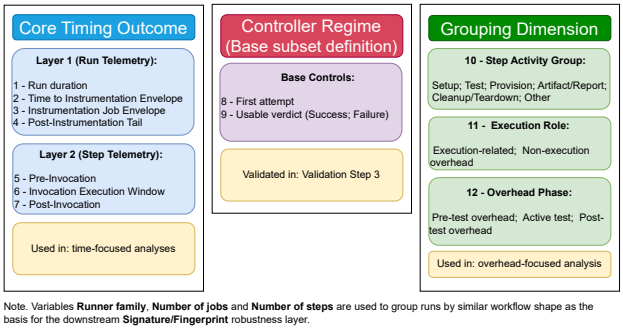

Fig. 3. Overview of the variables and their roles in the study. The timing decompositions are defined in Figure 2 and Section III-B. This figure relates the main timing outcomes, the Base controller regime, the overhead-grouping views, and the workflow-shape robustness layer to the study analyses.

c) Coverage snapshot.: The four-style analysis dataset contains **8,910** instrumentation-executed run×style records. Applying the first-attempt and usable-verdict controller regime yields the main analysis subset (**Base**) with **7,803** records. Coverage is effectively complete for **Layer 1**, because its timing components are derived from run/job-level telemetry. By contrast, **Layer 2** is observable for **5,116** Base records (**65.56%**), reflecting the more limited availability of directly observable step-level telemetry in public GitHub Actions repositories. Accordingly, Layer 2 analyses are conducted on the step-observable subset of Base, while the grouping dimensions remain available across the full four-style analysis dataset.

E. Analysis design and comparability regime

Because the study is observational, the analyses use a controlled comparison subset and a separate robustness layer. The controlled subset supports the main cross-style timing comparisons, while the robustness layer checks whether the same conclusions remain visible within structurally similar workflow settings.

a) Main study subset (Base).: The main timing analyses use a controlled subset restricted to the four study styles and to runs that are directly comparable. In practice, this means focusing on instrumentation-executed runs, first attempts, and usable run-level verdicts (`success` or `failure`). This subset is used for the main comparative analyses in the timing-focused research questions.

b) Workflow-shape fingerprint.: In addition to the main comparison subset, we use a workflow-shape fingerprint as a robustness layer. This fingerprint groups runs with broadly similar workflow structure so that later checks can be performed within more comparable workflow settings.

c) Role of the robustness layer.: The fingerprint is used in follow-up analyses to examine whether the main conclusions remain consistent within structurally similar workflow groups.

F. Statistical analysis procedures

Timing outcomes are heavy-tailed and often non-normal. Accordingly, we use non-parametric procedures for timing comparisons and emphasize effect sizes as indicators of practical impact.

a) Group comparisons and post-hoc interpretation (timing outcomes).: For overall multi-style comparisons, we use Kruskal–Wallis tests and report ϵ^2 as an effect size. For targeted pairwise interpretation, we apply Mann–Whitney tests with Holm correction for multiple comparisons and report Cliff's δ. For two-group regime contrasts, we use Mann–Whitney with Cliff's δ.

b) Predictability and tail summaries.: To characterize predictability and tail risk, we summarize dispersion using IQR and IQR/median, and tail behavior using P95 and P95/median when medians are non-zero. These summaries are reported alongside the non-parametric comparison results to separate typical performance from tail risk.

c) Categorical outcomes and proportions.: For categorical outcomes, we use contingency-table analysis with χ^2 tests and report **Cramér's** V as an effect size. For sparse cells, we interpret results cautiously and prioritize effect sizes and descriptive patterns. Where useful, we report two-sided confidence intervals for key proportions using established single-proportion methods.

G. Validation Procedure

Before turning to the RQ-specific analyses, we validate the study design through three complementary steps addressing technical internal consistency, source-grounded correctness, and conclusion stability. The goal is to confirm that the staged pipeline identifies the intended executions, that the timing measures behave coherently, that the main comparison regime is justified, and that the resulting conclusions are not driven by structurally unusual workflow settings.

Step 1: Automated rule-based validation

As the first validation step, we conduct an automated rule-based review of the analysis dataset to assess the internal consistency of the constructed records. This step serves as the study's technical data-quality validation layer: it verifies that the staged pipeline produces structurally valid, temporally coherent, and arithmetically consistent run×style observations before the RQ-specific analyses begin.

The automated validation comprises 53 rules spanning six aspects of dataset integrity: (1) structural integrity, including key uniqueness, required identifier presence, style-scope validity, and expected study flags; (2) temporal consistency, including ordering of run boundaries, invocation cutpoints, execution-end cutpoints, and execution-window bounds; (3) Layer 1 equation validity, including verification that the Layer 1 decomposition sums to total run duration; (4) Layer 2 recomputation validity, including verification that the pre-invocation, execution-window, and post-invocation components are non-negative, bounded by run duration, and consistent with the selected cutpoints and run boundaries; (5) source-traceability checks verifying that the selected invocation and execution-end cutpoints uniquely match the underlying step-level telemetry records; and (6) auxiliary consistency checks confirming that the derived workflow-shape, repeated-style,

TABLE I

SUMMARY OF STEP 1 AUTOMATED RULE-BASED VALIDATION. VALUES REPORT THE *average applicable-record accuracy across rules* WITHIN EACH VALIDATION CATEGORY. APPLICABLE-RECORD ACCURACY IS COMPUTED AS `OK / (OK + MISMATCH)`, EXCLUDING MISSING/NON-APPLICABLE CASES. OVERALL RATES ARE COMPUTED ON THE FULL DATASET ($n = 8{,}910$ RUN×STYLE RECORDS), AND BASE RATES ARE COMPUTED ON INSTRUMENTATION- EXECUTED FIRST-ATTEMPT USABLE-VERDICT RECORDS ($n = 7{,}803$).

Category	**What was checked**	**Overall**	**Base**
Structural integrity	Key uniqueness, required identifiers, style-scope validity, and expected study flags	100.00%	100.00%
Temporal consistency	Run-boundary ordering, execution-window ordering, cutpoint ordering, and window-within-run consistency	99.55%	100.00%
Layer 1 equation validity	Whether run duration equals the sum of the three Layer 1 components	100.00%	100.00%
Layer 2 recomputation validity	Whether the Layer 2 components are non-negative, bounded by run duration, and consistent with cutpoints and run boundaries	100.00%	100.00%
Source traceability	Whether selected invocation and execution-end cutpoints uniquely and correctly match the step-level source records	99.90%	99.90%
Auxiliary consistency	Whether the derived workflow-shape, repeated-style, and invocation-window support fields are present and internally coherent	100.00%	100.00%

and invocation-window support fields are present and internally coherent.

We report the automated checks both for the full dataset and for the study's main controlled timing regime, hereafter *Base*, defined as instrumentation-executed first-attempt records with usable verdicts (`success` or `failure`). This regime is used as the core technical-validation subset because it matches the timing comparison basis later used in RQ1–RQ3 while excluding non-usable outcomes that may truncate or otherwise distort performance-oriented comparisons. Table I summarizes the category-level results using average applicable-record accuracy across rules, computed as `ok / (ok + mismatch)` while excluding missing and non-applicable cases.

As shown in Table I, the automated validation indicates very strong internal consistency across all categories. Structural integrity, Layer 1 equation validity, Layer 2 recomputation validity, and auxiliary consistency are perfect in both the full dataset and Base. Temporal consistency is near-perfect overall and perfect in Base. Source traceability is likewise near-perfect, with the small residual deviations limited to unique-cutpoint selection and one invocation-step source-match rule rather than to broader inconsistencies in step-level traceability.

For telemetry-dependent rules, lower row-level support in the full dataset mainly reflects missing or non-applicable step telemetry rather than contradictory values. Accordingly, the category-level results in Table I are interpreted on an applicable-record basis for these categories. The full rule set, the explicit Layer 1 and Layer 2 equation checks, the interpretation of missing and non-applicable cases, and the detailed rule-level results are provided in Appendix IX-A, including Table IV. Taken together, Step 1 confirms the technical validity of dataset construction and the internal coherence of the timing measures used in the later analyses.

Step 2: Stratified manual audit

As the second validation step, we conduct a stratified manual audit of sampled run×style records against their source workflow-run evidence. Whereas Step 1 validates internal consistency at scale, Step 2 validates record-level correctness by checking whether sampled observations match the underlying GitHub Actions execution evidence.

The audited sample contains 124 records drawn from Base and from targeted supplementary strata designed to stress more fragile cases. The reviewed sample includes 60 core base-by-style records, together with additional Layer 2-observable cases, non-Base runs, complex same-style structures, edge cases, and multistyle executions. The final sample covers all four study styles.

For each sampled record, we manually check in-scope inclusion, instrumentation execution, style assignment, auditable Layer 1 and Layer 2 anchors, derived Layer 2 grouping labels, and the controller-facing fields used in the study, including first-attempt assignment and usable-verdict assignment. As in Step 1, telemetry-dependent checks are interpreted on an applicable-record basis, with accuracy computed as *correct* / (*correct* + *mismatch*) while excluding non-auditable cases.

The manual audit shows very strong agreement with the constructed dataset. In the full sampled set, all reviewed records are confirmed to be in scope, instrumentation-executed, correctly styled, and correct on the controller fields. Layer 1 boundaries are also correct for all auditable cases. For Layer 2, the grouping labels are correct for all auditable cases. The only residual disagreements are limited to three non-Base records affecting the Layer 2 invocation-anchor and execution-end-anchor checks, yielding an applicable-record accuracy of 96.39% (80/83) for each of those two anchor fields in the full sample.

These residual disagreements are confined to non-Base records where the sampling sheet still marked the run as not Layer 2 observable, while the current MainDataset now exposes measurable Layer 2 cutpoints for the same run. Thus, the detected mismatches reflect a narrow observability-state alignment issue between the sampling snapshot and the current

TABLE II
SUMMARY OF STEP 2 STRATIFIED MANUAL AUDIT RESULTS. ACCURACY IS REPORTED ON AN APPLICABLE-RECORD BASIS, COMPUTED AS *correct* / (*correct* + *mismatch*), EXCLUDING NON-AUDITABLE CASES.

Checked field	Full sample	Base
In-scope inclusion	100.00%	100.00%
Instrumentation executed	100.00%	100.00%
Style assignment	100.00%	100.00%
Layer 1 boundaries	100.00%	100.00%
Layer 2 invocation anchor	96.39%	100.00%
Layer 2 execution-end anchor	96.39%	100.00%
Step activity group	100.00%	100.00%
Execution role	100.00%	100.00%
Overhead phase	100.00%	100.00%
Attempt field	100.00%	100.00%
Usable-verdict assignment	100.00%	100.00%

dataset revision rather than broader errors in style detection, controller assignment, or grouping construction.

In Base, the sampled manual audit is fully clean: all auditable fields are confirmed as correct, including the Layer 2 anchor checks and the controller-facing assignments needed for the timing-comparison regime. Taken together, Step 2 provides strong source-grounded evidence that the constructed records and derived audit fields are highly accurate, and that any remaining disagreements are rare, localized, and outside the study's main comparison regime. Table II summarizes the field-level results for the full sample and for Base.

Step 3: Robustness check

As the third validation step, we conduct a robustness-check analysis to assess whether the study's main observational conclusions remain stable under adjacent comparison regimes and under constrained workflow-shape settings. Unlike Step 1, which validates internal consistency, and Step 2, which audits sampled-record correctness, Step 3 validates conclusion stability and the suitability of the study's main comparison basis.

The first part evaluates *controller-regime robustness*. The analysis compares the full executed four-style view, the first-attempt view, the study's controlled *Base* regime, and the adjacent rerun usable-verdict regime. Here, Base is defined by instrumentation-executed first attempts with usable run-level verdicts (`success` or `failure`). For each regime, the automated procedure summarizes the study timing measures by style and compares the resulting median-based style ordering against the reference ordering observed in Base. This design allows Step 3 to check both whether reruns form a distinct adjacent regime and whether retaining non-usable outcomes would alter the timing comparison basis. In the current study, usable-verdict first attempts provide the most appropriate controlled regime for timing-focused comparisons, while non-usable outcomes are retained only for the practice-facing analysis in RQ4.

To complement this ordering-based check, Step 3 also includes a focused statistical comparison between Base and the adjacent rerun usable-verdict regime. This confirms that reruns mainly alter the completion-oriented timing profile rather than the earliest entry into the instrumentation path. Thus, Step 3 provides both descriptive and inferential support for retaining the usable-verdict Base regime as the study's main timing comparison basis and for treating reruns as a separate adjacent condition.

The second part evaluates *workflow-shape robustness*. For this purpose, we use the workflow-shape fingerprint already defined in the study from runner family, executed-job-count bucket, and executed-step-count bucket. The automated procedure first checks exact-signature robustness by recomputing the same timing summaries within candidate signatures that have sufficient support and usable cross-style overlap. In the current snapshot, this exact-signature check is only partially confirmatory: only two exact signatures provide sufficiently interpretable cross-style overlap for deeper follow-up, and these do not yield a clean balanced four-style setting. Accordingly, exact signatures are retained only as a strict first-tier robustness lens.

To support later conclusion checks under still-limited workflow-shape variation but broader cross-style coverage, this first-tier exact-signature lens is complemented with a second-tier coarsened shape-family analysis. Thus, in the current study, signatures are used only as a downstream robustness device for follow-up validation, not as a primary controller or a parallel main analysis regime.

Appendix IX-A, including Table V, summarizes the supporting evidence for this robustness check. Overall, Step 3 confirms that Base is a justified main comparison regime and that the workflow-shape layer functions as a meaningful downstream robustness device rather than as a primary controller.

H. RQ specifications

This subsection specifies the analytical design for each research question in terms of objective, analysis regime, and outcomes. Timing components follow Figure 2 and Section III-B; controller and robustness regimes follow Section III-E; and statistical procedures follow Section III-F. Timing-focused analyses (RQ1–RQ3) use the controlled Base subset defined by instrumentation-executed first attempts with usable verdicts, whereas the practice-facing deployment and verdict-usability analysis (RQ4) uses instrumentation-executed first attempts without the usable-verdict restriction.

1) RQ1 specification: speed profiling:

a) Purpose.: RQ1 constructs the speed profile of the four emulator testing styles under the controlled Base regime.

b) Design.: Under the controller-defined Base subset, we compare styles on:

- **Run duration**,
- **Layer 1 components** (Time to Instrumentation Envelope, Instrumentation Job Envelope, Post-Instrumentation Tail),
- **Layer 2 components** (Pre-Invocation, Invocation Execution Window, Post-Invocation) where directly observable.

Layer 1 supports broad high-level speed comparisons across styles, while Layer 2 supports more precise invocation-centered interpretation on the step-observable subset.

c) Outcomes.: RQ1 identifies which styles are faster overall, which reach the instrumentation path earlier, which spend more time in the main execution window, and whether their speed advantage is offset by longer post-execution tails. These results form the speed-focused basis for the later style profile and decision-support synthesis.

2) RQ2 specification: predictability and tail risk:

a) Purpose.: RQ2 constructs a predictability profile for the four emulator testing styles.

b) Design.: Within the controller-defined Base subset, we evaluate predictability for run duration and the Layer 1/Layer 2 timing components using the dispersion and tail summaries defined in Section III-F. We then use selected fingerprint-qualified workflow families as a downstream validation layer to check whether key predictability conclusions remain visible inside structurally comparable groups.

c) Outcomes.: RQ2 identifies which styles are more stable, which are more tail-prone, and whether those predictability differences remain visible inside comparable workflow-shape settings. These results provide the predictability-oriented counterpart to the RQ1 speed profile.

3) RQ3 specification: overhead composition and actionable levers:

a) Purpose.: RQ3 explains *why* styles differ by attributing latency to timing subcomponents and overhead groupings, and by translating dominant overhead placements into actionable levers.

b) Design.: RQ3 treats **run duration** as the end-to-end baseline, uses Layer 1 to identify broad operational placement of instrumentation-related time, and then uses Layer 2 and the three grouping dimensions to interpret where overhead occurs and what type of overhead dominates. Overhead is analyzed through three complementary lenses:

- **timeline placement**: Pre-Invocation / Invocation Execution Window / Post-Invocation,
- **execution role**: Execution-related / Non-execution overhead,
- **activity group**: Setup / Provision / Test / Artifact-Report / Cleanup-Teardown / Other.

Dominant components are then translated into first-target optimization levers grounded in platform/provider guidance. These levers should be interpreted as plausible operational targets rather than experimentally validated remedies.

c) Outcomes.: RQ3 identifies the dominant overhead placements associated with each style and translates them into practice-facing optimization targets for the later style profile and synthesis.

4) RQ4 specification: deployment context and run-level verdict usability:

a) Purpose.: RQ4 complements the timing-based style profiles with two practice-facing dimensions: how styles are deployed across CI triggers and how often instrumentation-executed first-attempt runs yield a usable run-level verdict.

b) Design.: Using instrumentation-executed first-attempt run×style observations, we summarize:

- the distribution of CI trigger events,
- the distribution of run-level conclusions,
- the share of usable verdicts (success/failure) versus non-usable outcomes,
- and trigger-conditioned verdict patterns where relevant.

Triggers are interpreted as deployment-context markers rather than as causal drivers of performance.

c) Outcomes.: RQ4 identifies which styles are associated with specific CI deployment contexts, which styles are more likely to yield usable run-level verdicts, and whether any styles exhibit distinct trigger-conditioned verdict behavior.

IV. Empirical Analysis and Results

In this section, we present our empirical findings across the research questions. For each RQ, we describe the motivation, explain our analytical approach, and report the key findings.

RQ1: How do styles differ in developer feedback speed?

RQ1's design and statistical procedures are defined in Section III-H1. In this research question, we construct the speed profile of the four study styles under the controlled Base regime used for the main comparative analyses. Under the revised methodology, Base is defined by instrumentation-executed first-attempt run×style records with usable verdicts (`success` or `failure`), yielding **7,803** observations across the four study styles. The analysis is organized around the two timing layers defined earlier in the methodology: Layer 1 captures run-level timing around the instrumentation envelope, while Layer 2 decomposes the directly observable invocation path into pre-invocation, invocation-execution, and post-invocation segments. Together, these measures allow us to distinguish not only which style is faster overall, but also *where* that speed difference accumulates.

Across the usable-verdict Base regime, the overall style effect remains statistically significant for all seven timing outcomes, indicating that the four styles do not share the same operational speed profile. The strongest and most decision-relevant differences are summarized below.

a) Obs. 1.1 (Community is the fastest overall operational profile).: Community provides the clearest fast-path profile in the current usable-verdict Base. At the run level, it has the shortest median overall duration among the four styles, at **877.0s**, compared with **1,212.0s** for GMD, **1,817.0s** for Custom, and **1,864.5s** for Third-Party. This advantage is statistically significant relative to GMD ($p = 5.55 \times 10^{-12}$, Cliff's $\delta = -0.250$, small), Third-Party ($p = 8.37 \times 10^{-56}$, $\delta = -0.411$, small-to-medium), and Custom ($p = 3.58 \times 10^{-3}$, $\delta = -0.289$, small). The same pattern becomes even clearer in the main execution-oriented components. For the Layer 1 instrumentation-job envelope, Community records a median of **617.0s**, versus **1,208.0s** for GMD, **1,609.5s** for Custom, and **1,717.5s** for Third-Party; the contrasts are all statistically significant, with medium effects against GMD and large-leaning medium effects against Custom and Third-Party. Likewise, for the Layer 2 invocation-execution window, Community has the shortest median at **392.0s**, compared with **726.0s** for Custom, **1,076.0s** for GMD, and **1,881.5s** for Third-Party; these

contrasts are especially strong relative to GMD and Third-Party. This finding is also preserved in the two-tier robustness check: within the strongest exact-signature candidate shared by Community/GMD/Third-Party, and again in the corresponding coarsened shape-family view, Community remains faster than GMD and Third-Party on overall duration and on the main completion-oriented execution windows. Taken together, these results identify Community as the study's fastest overall operational profile, especially when speed is defined by the main execution windows rather than by initial entry alone.

b) Obs. 1.2 (GMD reaches the instrumentation path fastest, but does not finish fastest).: GMD shows the clearest *fast-entry* profile. In Layer 1, it reaches the instrumentation envelope earlier than the other styles, with a median time-to-envelope of **3.0s**, compared with **4.0s** for Community, **3.0s** for Custom, and **4.0s** for Third-Party. The difference relative to Community is highly significant ($p = 8.63 \times 10^{-29}$, $\delta = -0.398$, small-to-medium), while Custom is effectively tied with GMD on this earliest entry measure ($p = 0.722$). A similar advantage appears in Layer 2 pre-invocation, where GMD records a median of **79.5s**, compared with **131.5s** for Custom, **181.0s** for Community, and **561.0s** for Third-Party; the contrast against Third-Party is especially strong ($p = 6.30 \times 10^{-38}$, $\delta = -0.949$, very large). However, this early-start advantage does not translate into the fastest end-to-end completion. In the invocation-execution window, GMD remains much slower than Community (**1,076.0s** vs. **392.0s**, $p = 8.89 \times 10^{-22}$, $\delta = 0.485$, medium), and it also trails Community in total run duration. Under the two-tier robustness check, this fast-entry interpretation remains clearly supported relative to Third-Party in both the exact-signature and coarsened shape-family views. The corresponding Community-versus-GMD entry contrast becomes weaker inside the closest shared workflow-shape setting, so the robust interpretation is that GMD is the clearest *fast-entry* style overall, but not a universally dominant fastest-entry style in every within-shape comparison. Thus, GMD should be interpreted as a style that enters the instrumentation path quickly, but accumulates more time during the main execution phase than the fastest overall style.

c) Obs. 1.3 (Third-Party is the slowest sustained-execution profile).: Third-Party forms the clearest slow-path profile in the current Base regime. It has the longest median overall run duration (**1,864.5s**), the longest Layer 2 pre-invocation segment (**561.0s**), and the longest Layer 2 invocation-execution window (**1,881.5s**) among the four styles. Relative to GMD and Community, these differences are both statistically strong and practically meaningful. For example, Third-Party is substantially slower than Community in overall duration ($p = 8.37 \times 10^{-56}$, $\delta = -0.411$, small-to-medium) and even more clearly slower in the invocation-execution window ($p = 5.22 \times 10^{-29}$, $\delta = -0.612$, large). It also shows a very large disadvantage relative to GMD in pre-invocation time ($p = 6.30 \times 10^{-38}$, $\delta = -0.949$). Compared with Custom, Third-Party is similar in overall duration ($p = 0.149$) but remains markedly slower in the observable execution path, especially for pre-invocation ($p = 4.01 \times 10^{-9}$, $\delta = -0.700$) and invocation-execution ($p = 2.57 \times 10^{-7}$, $\delta = -0.613$). This slow-path interpretation is also robust: within the strongest exact-signature candidate and the matching coarsened shape-family view, Third-Party remains slower than both Community and GMD on overall duration and on the main execution-oriented components. This means that Third-Party is not merely delayed in one isolated phase; rather, it is persistently slower both before the observable invocation begins and during the main execution window itself. From a speed-profiling perspective, it is therefore the clearest candidate for the slowest sustained-execution style.

d) Obs. 1.4 (Custom shows a mixed speed profile: early entry comparable to GMD, mid-path performance between Community and Third-Party, and a very long completion tail).: Although Custom has smaller support in the current Base regime and should therefore be interpreted more cautiously, it still contributes a distinct speed profile. At entry, it behaves competitively: its median Layer 1 time-to-envelope is **3.0s**, matching GMD and improving on Community (**4.0s**; $p = 7.17 \times 10^{-3}$, $\delta = -0.264$, small), while remaining directionally faster than Third-Party (**4.0s**; $p = 0.119$). However, this early entry does not translate into a fast overall path. Its median run duration rises to **1,817.0s**, significantly slower than Community ($p = 3.58 \times 10^{-3}$, $\delta = 0.289$, small) and GMD ($p = 4.97 \times 10^{-7}$, $\delta = 0.530$, medium), and its Layer 1 instrumentation-job envelope is likewise much longer than Community's and GMD's. In Layer 2, Custom occupies a middle position in the core execution path: its median pre-invocation (**131.5s**) and invocation-execution window (**726.0s**) are clearly lower than Third-Party's (**561.0s** and **1,881.5s**) but do not match Community's fast-core profile. The clearest Custom-specific feature appears at the end of the observable path: its post-invocation median reaches **942.5s**, far above Community (**78.0s**), GMD (**4.0s**), and Third-Party (**5.0s**), with strong pairwise differences relative to GMD ($p = 1.91 \times 10^{-12}$, $\delta = -0.812$) and Third-Party ($p = 2.83 \times 10^{-7}$, $\delta = 0.602$), and a smaller but still significant difference relative to Community ($p = 1.09 \times 10^{-2}$, $\delta = 0.269$). Accordingly, Custom is best interpreted as a *mixed-speed* style: it can enter the instrumentation path early, and its core execution window is better than Third-Party's, but this advantage is offset by a substantially longer completion tail. The robustness evidence for Custom is narrower because it appears only in the second exact-signature candidate and the corresponding coarsened shape-family setting; within that narrower support, the same mixed profile remains directionally visible and should be treated as a supported but cautious feature of the four-style profile.

e) Obs. 1.5 (Community's speed advantage is concentrated in the core execution windows and comes with a longer tail).: Although Community is the fastest style overall, its advantage is not uniform across all timing segments. Its main strength lies in the execution-oriented middle of the run, but this is followed by a longer completion tail than the other major styles. In Layer 1 post-instrumentation tail, Community records a median of **6.0s**, compared with **1.0s** for GMD,

Custom, and Third-Party; the differences are highly significant relative to GMD ($p = 1.87 \times 10^{-51}$, $\delta = 0.531$), clear relative to Third-Party ($p = 3.96 \times 10^{-38}$, $\delta = 0.327$), and still significant relative to Custom ($p = 1.50 \times 10^{-5}$, $\delta = 0.418$). The same pattern is stronger in Layer 2 post-invocation, where Community has a median of **78.0s**, compared with **4.0s** for GMD, **5.0s** for Third-Party, and **942.5s** for Custom. Thus, Community's longer-tail pattern is specific to the two major alternative styles GMD and Third-Party rather than to all styles uniformly, because Custom exhibits an even heavier final tail. This tail-heavy completion pattern also remains visible in the two-tier robustness checks: within the strongest exact-signature candidate and in the corresponding coarsened shape-family view, Community continues to show a materially longer post-invocation segment than GMD and Third-Party. Accordingly, Community is best characterized not simply as "fast everywhere," but more precisely as a *fast-core/longer-tail* profile: it minimizes the main execution windows, yet still spends more time than GMD and Third-Party in the final completion segments after the observable execution work has ended.

RQ1 Summary (Speed profile). The study styles show distinct speed profiles under the controlled usable-verdict Base regime. Community is the fastest overall style, GMD shows the clearest fast-entry profile, and Third-Party is the slowest sustained-execution style. Custom contributes a mixed-speed profile: early entry is competitive, but overall completion is slowed by a much heavier tail. These patterns remain supported by the two-tier robustness checks, with stronger support for Community, GMD, and Third-Party and narrower but still usable support for Custom.

RQ2: Which styles are most predictable, and which expose developers to the greatest tail-latency risk?

RQ2's design and statistical procedures are defined in Section III-H2. In this research question, we characterize how predictable the four emulator testing styles are under the controlled Base regime. Under the revised methodology, Base is defined by instrumentation-executed first-attempt run×style records with usable verdicts (`success` or `failure`), yielding **7,803** observations across the four study styles. Beyond typical speed, the focus here is on *dispersion*, *tail exposure*, and *repeatability*: whether a style stays close to its own typical behavior or instead exhibits heavier variation and more pronounced upper-tail risk. As in RQ1, the analysis is organized around the two timing layers defined in the methodology. Layer 1 provides the broad run-level view, while Layer 2 helps locate whether unpredictability arises before invocation, during the main execution window, or after execution has ended.

Across the usable-verdict Base regime, the overall style effect is statistically significant for all seven predictability-oriented timing measures, indicating that the four styles do not share the same stability profile. The strongest and most decision-relevant findings are summarized below.

f) Obs. 2.1 (GMD is the most predictable style on the main completion-oriented measures).: GMD provides the clearest stability-first profile in the current usable-verdict Base. Using each style's median normalized absolute deviation from its own median as the main predictability-loss measure, GMD is consistently tighter than the other styles on the major completion-oriented outcomes. For example, its median normalized deviation is **0.061** for overall run duration, compared with **0.513** for Community, **0.231** for Custom, and **0.256** for Third-Party. The pairwise contrasts are highly significant, with a large practical difference relative to Community ($p = 2.66 \times 10^{-100}$, $|\delta| = 0.771$) and clearly meaningful differences relative to Custom ($p = 2.15 \times 10^{-10}$, $|\delta| = 0.669$) and Third-Party ($p = 7.95 \times 10^{-48}$, $|\delta| = 0.634$). The same pattern appears for the Layer 1 instrumentation-job envelope, where GMD again records the smallest normalized deviation (**0.060** versus **0.514** for Community, **0.283** for Custom, and **0.233** for Third-Party), and for the Layer 2 invocation-execution window, where GMD remains the tightest style (**0.045** versus **0.518** for Community, **0.680** for Custom, and **0.168** for Third-Party). Thus, although GMD is not the fastest overall style in RQ1, it is the most repeatable one on the main completion-oriented timing measures. This finding also remains directionally visible in the two-tier robustness checks, where GMD continues to appear as the tightest style in the strongest exact-signature setting and in the corresponding coarsened shape-family view.

g) Obs. 2.2 (Community is fast in typical terms, but predictability-poor).: Community shows the clearest speed–predictability tradeoff. In RQ1 it often provides the best median completion speed, but under predictability-oriented measures it exhibits much heavier variation than GMD and noticeably heavier variation than the more stable styles overall. Its median normalized deviation reaches **0.513** for run duration, **0.514** for the instrumentation-job envelope, **0.912** for pre-invocation, and **0.518** for the invocation-execution window. These values are all substantially larger than GMD's corresponding values, and the pairwise contrasts are consistently statistically significant with large effect sizes: $|\delta| = 0.771$ for run duration, $|\delta| = 0.837$ for the instrumentation-job envelope, $|\delta| = 0.789$ for pre-invocation, and $|\delta| = 0.823$ for the invocation-execution window. Even relative to Custom, Community is less stable on broad completion-oriented measures such as run duration ($p = 1.52 \times 10^{-3}$, $|\delta| = 0.315$) and the instrumentation-job envelope ($p = 3.79 \times 10^{-4}$, $|\delta| = 0.353$). Community is therefore best interpreted not as a uniformly strong style, but as a *fast but variable* one: it often minimizes the typical execution time, yet it is much less stable once dispersion and tail exposure are considered. This interpretation also remains directionally visible under the two-tier robustness checks, where Community continues to appear less predictable than GMD in both the exact-signature and coarsened shape-family views.

h) Obs. 2.3 (Third-Party carries the strongest absolute tail-risk profile).: Third-Party is the clearest tail-risk-sensitive style in the current Base regime. Its medians are already large on the major completion-oriented measures, and its upper-tail exposure is substantial in absolute wall-clock

terms. In particular, Third-Party records a median overall duration of **1,864.5s**, a median instrumentation-job envelope of **1,717.5s**, and a median invocation-execution window of **1,881.5s**; the corresponding P95 values remain operationally large, including **34,603,064.5s** for total run duration, **3,804.0s** for the instrumentation-job envelope, and **2,305.1s** for the invocation-execution window. Relative to GMD, Third-Party is both slower and less predictable on these measures, with statistically significant differences and meaningful effect sizes. For example, its median normalized deviation is **0.256** versus **0.061** for run duration ($p = 7.95 \times 10^{-48}$, $|\delta| = 0.634$), **0.233** versus **0.060** for the instrumentation-job envelope ($p = 2.64 \times 10^{-44}$, $|\delta| = 0.609$), and **0.168** versus **0.045** for the invocation-execution window ($p = 3.93 \times 10^{-15}$, $|\delta| = 0.579$). Relative to Community, Third-Party is not always worse in *normalized* variability, but this does not make it operationally safer, because its absolute completion times are already much larger. Thus, Third-Party is best interpreted as the strongest *absolute tail-risk* profile: when long completion delays are costly, it is the clearest style to avoid. This pattern also remains directionally consistent with the two-tier robustness checks.

i) Obs. 2.4 (Custom has a mixed predictability profile and should be interpreted cautiously).: Custom remains part of the study profile, but its predictability signal is mixed and should be interpreted with caution because of its smaller support. In Layer 1, Custom appears more stable than Community on broad completion-oriented measures, including run duration and the instrumentation-job envelope. Its median normalized deviation is **0.231** for run duration versus **0.513** for Community ($p = 1.52 \times 10^{-3}$, $|\delta| = 0.315$), and **0.283** for the instrumentation-job envelope versus **0.514** for Community ($p = 3.79 \times 10^{-4}$, $|\delta| = 0.353$). However, this advantage does not generalize cleanly into Layer 2. On the directly observable invocation-centered measures, Custom shows a weaker and less favorable predictability signal: its median normalized deviation is **0.570** for pre-invocation and **0.680** for the invocation-execution window, both substantially larger than GMD's (**0.107** and **0.045**) and also larger than Third-Party's (**0.062** and **0.168**), with statistically significant pairwise differences where support is available. Thus, Custom is best characterized as a *mixed profile*: it shows moderate stability in the broad Layer 1 view, but a weaker and less favorable predictability signal in the more precise Layer 2 view. The same general pattern remains directionally visible in the second robustness tier, but because the available support for Custom is limited, this finding should be treated as a supported but cautious profile feature rather than as a dominant headline result.

RQ2 Summary (Predictability and tail risk). The study styles show distinct predictability profiles under the controlled usable-verdict Base regime. GMD is the most predictable style, with the tightest dispersion and lightest relative tails on the main completion-oriented measures. Community often delivers better typical speed, but with much heavier variability. Third-Party shows the strongest absolute tail-risk profile, while Custom presents a mixed and more cautious profile. Together, these findings complement the RQ1 speed profile and support later style-level decision guidance.

RQ3: What are the dominant sources of overhead by style, and which levers best improve developer feedback KPIs?

RQ3's design and statistical procedures are defined in Section III-H3. In this research question, we characterize where overhead accumulates across the four emulator testing styles under the controlled Base regime. Under the revised methodology, Base is defined by instrumentation-executed first-attempt run×style records with usable verdicts (`success` or `failure`), and the Layer 2 overhead analysis is conducted on the **5,116** Base records for which the step-level path is directly observable. The goal is not only to measure how much time is spent, but also to identify *where* that time is placed across the observable execution path: before invocation, during the main execution window, or after observable execution has ended.

Across the Base step-observable subset, the overall style effect is statistically significant for all three Layer 2 placement shares, indicating that the four styles do not distribute observable time in the same way. The strongest and most decision-relevant findings are summarized below.

j) Obs. 3.1 (GMD is the clearest execution-centric style).: GMD provides the clearest execution-centric overhead profile in the usable-verdict Base. Its median invocation-execution-window share reaches **92.7%**, compared with **38.5%** for Community, **35.9%** for Custom, and **70.1%** for Third-Party. The contrast is especially strong relative to Community ($p = 2.15 \times 10^{-45}$, Cliff's $\delta = -0.715$, large) and Third-Party ($p = 2.91 \times 10^{-29}$, $\delta = 0.828$, large). At the same time, GMD has the smallest post-invocation share, at only **0.4%**, with very strong separation from Community (**8.0%**; $p = 1.62 \times 10^{-55}$, $\delta = 0.794$, large) and Custom (**56.4%**; $p = 1.79 \times 10^{-10}$, $\delta = 0.746$, large). Thus, GMD is best characterized as the clearest *execution-centric* style: when it is slow, the first optimization target is the execution path itself rather than the completion tail.

k) Obs. 3.2 (Third-Party is best characterized by heavy entry and heavy execution, not by a dominant completion tail).: In the updated Base, Third-Party is not best understood as a tail-dominated style. Instead, its observable delay is concentrated before invocation and inside the execution window. It records the largest median pre-invocation time at **561.0s**, compared with **79.5s** for GMD, **131.5s** for Custom, and **181.0s** for Community, and it also has the largest median invocation-execution window at **1,881.5s**. In share terms, its observable path is dominated by pre-invocation (**23.3%**) and execution (**70.1%**), while its post-invocation share remains very small (**0.3%**). These contrasts are statistically strong, especially relative to GMD for pre-invocation share (**23.3%** vs. **6.8%**, $p = 7.16 \times 10^{-27}$, $\delta = -0.791$, large) and relative to Community for execution-window share (**70.1%** vs. **38.5%**, $p = 9.77 \times 10^{-8}$, $\delta = -0.292$, small). Thus, Third-Party is

best interpreted as a *heavy-entry plus heavy-execution* style rather than as a completion-tail-dominated one.

l) Obs. 3.3 (Community has a distributed overhead profile rather than a single dominant overhead source).: Community does not concentrate observable time in one component. Instead, it exhibits a more distributed overhead profile across the step-observable path, with median shares of **16.5%** in pre-invocation, **38.5%** in the execution window, and **8.0%** in post-invocation. Relative to GMD, Community has much less execution concentration (**38.5%** vs. **92.7%**, $p = 2.15 \times 10^{-45}$, $\delta = -0.715$, large) and a substantially larger residual tail (**8.0%** vs. **0.4%**, $p = 1.62 \times 10^{-55}$, $\delta = 0.794$, large). Relative to Third-Party, it has a smaller entry burden (**16.5%** vs. **23.3%**, $p = 4.09 \times 10^{-8}$, $\delta = -0.300$, small) and a smaller execution-window share (**38.5%** vs. **70.1%**, $p = 9.77 \times 10^{-8}$, $\delta = -0.292$, small), but a much larger post-invocation share (**8.0%** vs. **0.3%**, $p = 8.35 \times 10^{-38}$, $\delta = 0.703$, large). Thus, Community is best characterized as a *distributed* overhead profile: entry, execution, and residual completion all matter, so optimization should not focus on only one source.

m) Obs. 3.4 (Custom remains a cautious, tail-heavy mixed case).: Custom remains part of the four-style overhead profile, but it should be interpreted more cautiously because of its smaller support. In the usable-verdict Base Layer 2 view, it shows a mixed overhead composition: a relatively small pre-invocation share (**7.4%**), a moderate execution-window share (**35.9%**), and a very large post-invocation share (**56.4%**). This makes Custom the clearest tail-heavy case in the pooled view. Its post-invocation share is far larger than GMD's (**0.4%**; $p = 1.79 \times 10^{-10}$, $\delta = 0.746$, large) and Third-Party's (**0.3%**; $p = 8.53 \times 10^{-11}$, $\delta = 0.772$, large), and it is also larger than Community's (**8.0%**; $p = 1.64 \times 10^{-2}$, $\delta = -0.254$, small). At the same time, its execution-window share remains well below GMD's (**35.9%** vs. **92.7%**, $p = 1.18 \times 10^{-14}$, $\delta = -0.903$, large) and also below Third-Party's (**35.9%** vs. **70.1%**, $p = 1.83 \times 10^{-4}$, $\delta = -0.445$, small-to-medium). Accordingly, Custom is best interpreted as a *tail-heavy mixed case*: its observable overhead is not dominated by entry or execution, but by a large residual completion segment. Because support remains limited, this should be treated as a cautious but still meaningful feature of the four-style profile.

RQ3 Summary (Overhead composition). The study styles show distinct overhead-placement profiles under the controlled usable-verdict Base regime. GMD is the clearest execution-centric style, Third-Party is best characterized by heavy entry and heavy execution, and Community shows a more distributed overhead profile. Custom remains a cautious, tail-heavy mixed case. Together, these findings identify the main optimization targets later used in the style profile and decision-support synthesis.

RQ4: How do emulator testing styles differ in CI deployment context and run-level verdict usability?

RQ4's design and statistical procedures are defined in Section III-H4. In this research question, we complement the timing-based style profiles with two practice-facing dimensions: how the four emulator testing styles are deployed across CI trigger contexts, and how often their instrumentation-executed first-attempt runs yield usable and successful run-level outcomes. Unlike the timing-focused research questions, RQ4 is not restricted to the usable-verdict Base subset, because the aim here is to study verdict usability itself. Accordingly, the analysis uses instrumentation-executed first-attempt run×style observations and examines run-level outcomes and trigger distributions across Community, Custom, GMD, and Third-Party.

Across this first-attempt regime, the overall style effect is statistically significant for the main verdict- and deployment-context measures. The strongest and most decision-relevant findings are summarized below.

n) Obs. 4.1 (Styles differ in usable run-level verdict rates, although the overall separation is modest).: The four styles differ significantly in how often they produce a usable run-level verdict, defined here as a terminal `success` or `failure` outcome rather than a cancelled or otherwise non-usable result. Under the first-attempt regime, GMD records the highest usable-verdict rate at **99.2%**, followed by Third-Party at **98.1%**, Community at **89.2%**, and Custom at **87.2%**. The overall association between style and usable-verdict status is statistically significant, with a small but non-trivial effect size. Pairwise contrasts show especially clear separation between Community and GMD, Community and Third-Party, and between Custom and the two higher-usable styles. Thus, while usable-verdict behavior is not the most strongly separating style dimension in the study, it does add a practical distinction: GMD and Third-Party are more likely than Community and Custom to end in an actionable terminal run outcome.

o) Obs. 4.2 (Among usable verdicts, styles differ sharply in success rate).: Conditioning only on runs that reach a usable verdict reveals a much stronger and more practically meaningful separation. Among usable verdicts, GMD yields **98.1% success**, Community yields **79.1% success**, Custom yields **23.5% success**, and Third-Party yields only **16.5% success**. The resulting style-by-verdict association is extremely strong, with a moderate-to-large effect size and highly significant pairwise differences across the main style contrasts. In practical terms, GMD is not only highly likely to yield a usable verdict; it is also overwhelmingly success-oriented once a usable verdict is produced. By contrast, Third-Party and Custom are both strongly failure-dominant, while Community occupies an intermediate position. This makes run-level verdict behavior a useful practice-facing complement to the timing profile: styles differ not only in speed and predictability, but also in the success rate of the actionable outcomes they tend to produce.

p) Obs. 4.3 (Styles are deployed in markedly different CI trigger contexts).: The four styles are also associated with sharply different CI deployment contexts. Community is used predominantly under `push` events, accounting for **88.1%** of its first-attempt observations. By contrast, GMD and Third-Party are both overwhelmingly associated with `schedule` events, at **90.2%** and **87.3%**, respectively. Custom differs

from all other styles in being most strongly associated with `pull_request`, which accounts for **66.7%** of its observations. The overall style-by-event association is highly significant and exhibits a moderate-to-large effect size, indicating that deployment context is one of the clearest non-timing distinctions among the styles. Accordingly, the styles should not be interpreted as interchangeable CI choices used in the same operational setting: they are embedded in meaningfully different trigger regimes in practice.

q) Obs. 4.4 (For Third-Party and Custom, success rate is strongly trigger-conditioned).: For some styles, especially Third-Party and Custom, verdict behavior is not only style-specific but also strongly associated with deployment context. In Third-Party, usable verdicts under `push` are almost always successful, whereas under `schedule` they are overwhelmingly failures; this contrast is statistically extreme and operationally important. Custom shows a similar, though smaller-support, pattern: under `pull_request`, usable verdicts are mostly failures, while under `push` they are much more often successful. GMD also exhibits event-conditioned verdict variation, but within a much healthier success-oriented range. These findings indicate that verdict usability and success rate should not be treated as purely style-intrinsic. For at least some styles, especially Third-Party and Custom, deployment trigger and verdict behavior are closely linked. This makes deployment context an important part of the later style profile and decision-support interpretation.

RQ4 Summary (Deployment context and run-level verdict usability). The study styles differ not only in timing behavior, but also in how they are deployed and in the run-level outcomes they produce. GMD and Third-Party are the most likely to yield a usable verdict, but among usable outcomes GMD is strongly success-dominant whereas Third-Party and Custom are strongly failure-dominant. Community is mainly associated with `push`, GMD and Third-Party with `schedule`, and Custom with `pull_request`. For Third-Party and Custom in particular, success rate is strongly trigger-conditioned. Together, these findings add a practice-facing deployment and actionability dimension to the study's style profiles.

V. Operational performance profile summary

This section synthesizes the findings from RQ1–RQ4 into an operational performance profile for each execution style and derives a practitioner-facing decision-support guide from that profile. Unless stated otherwise, conclusions refer to executed emulator run×style observations under the **Step 3-validated controlled regime** of **usable-verdict** and **first-attempt** runs, with workflow-shape fingerprints used only for **robustness confirmation**. RQ4 complements the timing-focused profile (RQ1–RQ3) by characterizing **run-level verdict usability**, **success rate**, and **deployment context** (how styles are actually deployed across triggers). The operational profile is the primary synthesis outcome; the decision-support guide is a **profile-derived, evidence-grounded practitioner aid**, not a learned or externally validated recommender. :contentReference[oaicite:0]index=0

a) Key cross-style implications.: **Community** remains the practical default with broad coverage and the fastest typical end-to-end completion. Its main operational tradeoff is not typical speed, but **predictability**: it is the fastest overall style, yet also the most tail-exposed after normalization. RQ3 further shows that Community has a **distributed overhead profile**, with meaningful entry cost, meaningful execution cost, and a non-trivial residual tail rather than a single dominant overhead source. RQ4 complements this picture by showing that Community is predominantly deployed on **push**-triggered runs and usually yields usable verdicts with a comparatively healthy success rate, reinforcing its role as the general-purpose baseline.

GMD provides the strongest **predictability-first** profile. It shows the clearest fast-entry behavior, the most stable completion-oriented timing profile, and the most strongly **execution-centric** overhead structure. Its main tradeoff is that completion can still be slower than Community because much of its runtime is concentrated inside a comparatively long but bounded execution window. RQ4 indicates that GMD is most often deployed under **scheduled** execution contexts and overwhelmingly produces usable, success-oriented outcomes, reinforcing its suitability for predictable, repeatable feedback loops.

Third-Party combines the slowest sustained-execution behavior with the strongest **absolute tail-risk** profile. The updated RQ3 results show that its dominant penalty is no longer best understood as a post-execution completion tail; instead, it is best characterized by **heavy entry plus heavy execution**. RQ4 further shows that Third-Party is strongly **trigger-conditioned**: it is mainly deployed under **schedule**, where usable outcomes are overwhelmingly failure-oriented, while the much smaller **push** regime is strongly success-oriented. Operationally, Third-Party should therefore be treated as a context-sensitive style whose main risks lie in entry/provisioning delay, provider-side execution cost, and trigger policy rather than in a single completion-tail bottleneck.

Custom remains too sparse for broad generalization, but the observed evidence still supports a distinct, cautious profile. In RQ1, Custom appears as a **mixed-speed** style: early entry is competitive, the core execution path is better than Third-Party's, but overall completion is slowed by a much heavier tail. In RQ2 and RQ3, it also appears as a **mixed and tail-heavy** case, with moderate stability in the broad Layer 1 view but weaker Layer 2 stability and a large post-invocation share. RQ4 suggests that Custom is disproportionately associated with **pull_request** contexts and has a low success rate in that regime. Accordingly, Custom should be treated as an exception path driven by repository-specific orchestration constraints rather than as a general default.

A. Profile-derived decision-support guide

Beyond summarizing profiles, we translate RQ1–RQ4 into a lightweight decision-support guide. The guide maps **prac-**

TABLE III
COMPACT OPERATIONAL PERFORMANCE PROFILE BY STYLE UNDER THE STEP 3-VALIDATED CONTROLLED REGIME. "SPEED PROFILE" SUMMARIZES RQ1; "PREDICTABILITY" SUMMARIZES RQ2; "OVERHEAD SOURCE & LEVER" SUMMARIZES THE DOMINANT OPTIMIZATION TARGET INFERRED FROM RQ3; "VERDICT & DEPLOYMENT" SUMMARIZES RQ4. LEVERS ARE *first optimization targets*, NOT EXPERIMENTALLY VALIDATED REMEDIES.

Style	Speed profile	Predictability	Overhead source & lever	Verdict & deployment (RQ4)
Community	Fastest overall completion; strong fast-core profile, but not uniformly fast across all phases because the completion tail remains non-trivial.	Fast but variable; the most tail-exposed style after normalization on the main completion-oriented measures.	Distributed overhead profile across entry, execution, and residual tail. **First optimization target:** stabilize setup/entry and reduce execution-path cost, then inspect remaining tail contributors.	Usually yields usable verdicts and a comparatively healthy success rate; most often deployed on **push**.
Custom	Mixed-speed case: early entry is competitive, mid-path execution is better than Third-Party's, but overall completion is slowed by a very heavy tail.	Mixed and cautious profile: moderate Layer 1 stability, weaker Layer 2 stability, and small support.	Tail-heavy mixed case with large post-invocation share. **First optimization target:** reduce bespoke completion-tail work and standardize custom orchestration where feasible.	Sparse, low-success profile in the observed sample; appears most often on **pull_request**; interpret cautiously.
GMD	Clearest fast-entry profile; slower than Community in end-to-end completion, but stable and execution-focused.	Most predictable style on the main completion-oriented measures; tight dispersion and light relative tails.	Execution-centric, near-zero residual tail. **First optimization target:** optimize the execution path itself (test efficiency, parallelization, flake reduction, execution simplification).	Most often yields usable and successful outcomes; mainly deployed on **schedule**.
Third-Party	Slowest sustained-execution style, with the heaviest entry burden and the longest execution window.	Strongest absolute tail-risk profile, even when normalized variability is not always the worst.	Heavy entry plus heavy execution, not primarily completion-tail dominated. **First optimization target:** reduce entry/provisioning delay and shorten provider-side execution cost; treat trigger policy separately.	Usually yields a usable verdict, but among usable outcomes is strongly failure-dominant overall; mainly deployed on **schedule** and strongly trigger-conditioned.

titioner objective (predictable feedback vs. fast first signal vs. fast typical completion vs. usable and successful run outcomes) together with **feasibility constraints** (e.g., whether GMD or an external-provider workflow is adoptable) to a recommended style. Recommendations are paired with the **dominant optimization target** inferred from RQ3 and refined by the **deployment/outcome context** signals in RQ4. These targets are included as evidence-grounded starting points for optimization, not as experimentally validated fixes.

a) Why a profile-derived guide rather than a learned tree.: While decision-tree learning (e.g., CART) can produce human-readable rules, our corpus is strongly skewed toward Community-style executions, and several styles remain sparse in absolute support. Under such imbalance, a learned classifier can degenerate into recommending the majority style across most branches, limiting practical utility. Therefore, we present the decision-support structure as a **profile-derived guide** grounded in the empirical findings from RQ1–RQ4, and leave automated training and validation of a recommender model (including imbalance-aware learning, intervention testing, and external validation) as future work.

b) Interpretation and scope.: The guide reflects the main empirical patterns established in the pooled controlled regime. **GMD** is preferred when **predictability**, **fast entry**, or **high-confidence successful outcomes** are the primary objective and adoption is feasible, and it is commonly deployed in scheduled regression contexts. **Community** remains the **general-purpose baseline**, with the fastest typical completion and broad practical coverage, but also the greatest normalized tail exposure, making setup stabilization and execution-cost reduction the main first optimization targets. **Third-Party** is appropriate when **external execution capacity** or provider-specific workflows are required, but it is strongly trigger-conditioned and, when executed, its dominant penalties lie in entry/provisioning and execution cost rather than in a single completion-tail bottleneck. **Custom** remains an exception path for repository-specific constraints rather than a generally recommended default. Accordingly, the operational profile should be interpreted as the study's main practical synthesis, while the decision-support guide should be interpreted as an **evidence-grounded practitioner aid** derived from that profile rather than as a learned, externally validated, or interventionally confirmed decision policy.

VI. IMPLICATIONS

Our findings have implications for (i) practitioners who must select, operate, and optimize instrumentation-testing setups under CI constraints—including app developers, QA/SDETs, DevOps/release owners, open-source maintainers, and CI cost owners—(ii) researchers studying CI performance and operational reliability, and (iii) CI platforms, third-party providers, and tooling/action authors who can reduce sources of avoidable overhead, tail risk, and low-value outcome instability.

A. Implications for Practitioners

Choose styles by objective, then optimize the dominant phase. Instrumentation-test execution styles are not interchangeable: they trade off typical speed, predictability, overhead placement, and run-level outcome quality. Practitioners should first decide whether the primary objective is (a)

Decision-support guide (profile-derived).

- **If the primary objective is predictable feedback:**
  - **If GMD is feasible:** recommend **GMD**.
    Primary optimization target: improve execution efficiency inside the main execution path.
  - **Otherwise:** use **Community** as the practical fallback, while treating it as a higher-tail-risk option.
    Primary optimization target: reduce entry/setup variability and execution-path cost.
- **If the primary objective is fast first signal:**
  - **If GMD is feasible:** recommend **GMD**.
  - **Otherwise:** recommend **Community**.
    Primary optimization target: reduce entry/setup instability and orchestration variance.
  - **Do not prefer Third-Party on early-feedback grounds alone**, because its main penalties lie in slow entry and long execution.
- **If the primary objective is fastest typical end-to-end completion:**
  - **Default recommendation: Community**.
  - **If predictability matters nearly as much as speed and GMD is feasible:** prefer **GMD** as the safer tradeoff.
  - **Use Third-Party mainly when external execution capacity or provider-specific constraints are required**.
    Primary optimization target: reduce entry/provisioning delay and provider-side execution cost.
- **If the objective includes usable and successful run outcomes:**
  - Prefer **GMD** when feasible; it combines the highest usable-verdict rate with the strongest success rate.
  - Use **Community** as the general-purpose fallback with broader practical coverage and a substantially healthier success profile than Third-Party or Custom.
  - Treat **Third-Party** and **Custom** as strongly context-sensitive: confirm trigger policy and evaluate their success behavior in the relevant event regime before relying on them.
- **If the dominant observed problem is overhead placement rather than typical median speed:**
  - **For Community:** treat the problem as **distributed**; inspect entry/setup, execution, and residual tail contributors together.
  - **For GMD:** treat the problem as primarily **execution-centric**; focus on the execution path itself.
  - **For Third-Party:** treat the problem as **entry + execution**; inspect provisioning/orchestration delay and provider-side execution cost.
  - **For Custom:** treat the problem as primarily **tail-heavy and bespoke**; reduce post-execution cleanup/reporting and standardize custom orchestration.
- **Custom as exception:** recommend **Custom** only when repository-specific constraints prevent adopting reusable Community/GMD components or provider workflows. When Custom is used, prioritize standardization, script reduction, and reuse of shared setup/orchestration components.

Fig. 4. Profile-derived decision-support guide. Recommendations map practitioner objective and feasibility constraints to a suggested style, paired with the dominant optimization target inferred from the pooled overhead analysis (RQ3) and refined by deployment/outcome context (RQ4). The guide is intended as an interpretable, evidence-grounded aid rather than as a validated recommender.

minimizing typical feedback time, (b) improving predictability and reducing tail risk, (c) targeting the dominant overhead source, or (d) increasing the likelihood of usable and successful outcomes in the relevant CI context. Once a style is selected, optimization should target the empirically dominant phase rather than indiscriminately "speeding up CI."

Treat setup and provisioning as first-class performance risks (especially for PR gating). Across styles, setup/provisioning and early orchestration can dominate both latency and variability. This suggests that common tactics—caching, minimizing repeated environment bootstraps, prebuilding emulator images, eliminating redundant dependency installation, and reducing unnecessary job fan-out—can yield larger and more reliable gains than micro-optimizing test code alone. In particular, when the bottleneck lies before the main execution window, test-level optimizations alone will not reduce end-to-end time or tails.

Use step-level traces to localize bottlenecks and avoid misattribution. Run-level completion time is useful for high-level monitoring, but it can be misleading for diagnosis. Step-level telemetry supports decomposition into entry/setup, main execution, and post-execution phases, enabling teams to (i) distinguish infrastructure/provisioning delay from test execution cost, (ii) anchor attribution to the actual invocation path, and (iii) verify whether an intervention improved the intended phase rather than merely shifting time elsewhere.

Align regime to purpose: PR feedback, nightly regression, and scheduled monitoring require different tradeoffs. Operational regimes matter. A setup that performs acceptably in scheduled regression may be unsuitable for pull requests if it produces long tails, low success rates, or high cancellation burden. Conversely, a configuration optimized for fast PR feedback may require additional stabilization before it becomes reliable at scale for scheduled or release-oriented monitoring. Release owners can use predictability, usable-verdict rate, and success-rate signals to define gates that balance delivery risk against feedback cost.

Operationalize "predictable CI" as a management and cost objective, not only a technical metric. Tail-heavy configurations translate into developer waiting time, reruns, and wasted runner minutes. Engineering managers and CI cost owners can use tail metrics, phase attribution, and outcome rates to prioritize investments such as reducing provisioning churn, bounding provider queues, stabilizing entry/setup, or shrinking the execution window. These signals also help justify when managed or external execution solutions are worth their cost.

Use deployment context as part of style selection, not only as background metadata. The same style can behave differently across trigger contexts. In particular, some styles are strongly associated with specific event regimes, and their success behavior can shift sharply by trigger. Practitioners should therefore evaluate style choice in the actual CI context where it will be used—for example, `push`, `pull_request`, or `schedule`—rather than assuming that pooled results transfer uniformly across deployment settings.

Adopt workflow-shape fingerprints as a lightweight robustness check in CI analytics and monitoring. Performance comparisons can be dominated by workflow structure rather than by the testing technique itself. Our workflow-shape fingerprint and signature hash provide a lightweight method to group comparable executions and verify that conclusions are not driven by atypical pipelines. Practitioners can use

fingerprints to (i) track regressions within stable workflow regimes as workflows evolve, and (ii) compare alternative implementations of the same testing intent under more comparable structural conditions.

Make style choices maintainable for open-source and small teams. For maintainers operating under limited CI expertise and volunteer time, the most robust choice may not be the fastest median. Style selection should account for maintenance burden, failure-triage effort, trigger sensitivity, and the ease of reproducing failures locally. The decision-support guide can be used to select approaches that remain stable under changing dependencies and runner environments rather than chasing speed alone.

B. Implications for Researchers

CI performance studies benefit from regime-aware designs. Our results reinforce that CI outcomes are regime-dependent: trigger context, reruns/attempts, and usable-verdict filtering can shift both timing distributions and practical outcome profiles. Observational comparisons should therefore avoid pooling heterogeneous regimes by default and should report the comparison basis explicitly.

Workflow structure is a confounder that should be modeled, not ignored. The workflow-shape fingerprint operationalizes a broader point: workflow configuration and orchestration complexity often co-determine outcomes. Treating workflow shape as a first-class covariate, matching layer, or robustness device improves interpretability and reduces the risk of spurious style-level claims.

Step-level telemetry enables explanation, not just measurement. CI telemetry is increasingly rich. Using step-level traces as an explanation layer provides a bridge between descriptive timing results and actionable recommendations, and supports more credible replication across projects and platforms that expose similar traces.

Outcome quality should be studied together with performance. The study shows that timing results alone are not sufficient for operational interpretation. Styles can differ not only in speed and predictability, but also in usable-verdict rates, success rates among usable runs, and trigger-conditioned outcome behavior. Future CI studies should therefore consider combining performance and outcome-quality measures rather than treating them as unrelated dimensions.

Datasets and signatures can support replication and drift studies. The signature-hash abstraction offers a practical handle for replication and longitudinal analyses: researchers can re-sample comparable workflow shapes over time to study workflow evolution, performance drift, and the stability of style-level conclusions.

C. Implications for CI Platforms and Tool Providers

Expose stable, standardized phase markers for performance attribution. A recurring challenge is inconsistent naming and structure of test-invocation steps across repositories. Platform-level support for standardized phase markers—for example, explicit test execution boundaries, provisioning categories, and provider handshake phases—would improve both research and operational analytics.

Reduce and stabilize configuration/setup overhead. Many bottlenecks arise before tests start. Improvements in emulator boot performance, caching primitives, and declarative environment reuse can directly reduce tails and increase predictability. Clearer guidance and automation for optimal caching, emulator snapshots, and environment reuse would reduce practitioner burden and make good defaults more accessible to smaller teams.

Improve provider/device-farm observability and reliability. When third-party providers dominate latency or failure-oriented outcomes, better observability—for example queue time, device allocation delay, provider-side execution timing, and trigger-conditioned health reporting—would enable more targeted mitigation and fairer comparisons across providers. Provider APIs that emit phase breakdowns aligned with CI telemetry would materially improve debuggability.

Support action/tool authors with performance transparency. Community action authors and Gradle/tooling maintainers can use phase-level evidence to identify overhead introduced by setup scripts, dependency installation, emulator configuration, and artifact/reporting steps. Publishing performance characteristics and recommended configurations—for example caching, snapshot support, concurrency limits, and known trigger-sensitive behaviors—can reduce downstream variability and improve predictability across repositories.

VII. Threats to Validity

We discuss threats to construct, internal, external, and conclusion validity in light of the current study design, following established empirical software engineering guidance [31, 32].

A. Construct Validity

a) Constructing timing outcomes from CI telemetry.: Our timing outcomes are derived from GitHub Actions run/job/step telemetry rather than from raw execution logs. This provides a scalable and structured basis for historical analysis, but it also means that the study depends on the semantic accuracy of API-exposed timestamps and boundaries. In particular, the Layer 1 and Layer 2 decompositions rely on the assumption that the selected cut points correspond to the intended execution phases. If GitHub timestamps are delayed, missing, or inconsistent for some runs, measured durations may deviate from the underlying execution reality. We mitigate this threat by validating temporal coherence, decomposition equations, and source traceability in Step 1, and by auditing the selected anchors manually in Step 2.

b) Invocation anchoring and execution-end selection.: The Layer 2 decomposition depends on correctly identifying the matched invocation step and the last execution-related step. This is a construct-level challenge because repositories can encode instrumentation execution through repository- specific scripts, wrappers, composite actions, or provider-side handoffs. A misidentified invocation start or execution-end

boundary could shift time between pre-invocation, invocation-execution, and post-invocation segments. We mitigate this threat by using style-aware matching rules, preferring explicit instrumentation-test invocations when available, validating anchor traceability in Step 1, and performing source-grounded manual checks in Step 2.

c) Overhead grouping and category assignment.: RQ3 interprets Layer 2 timing through grouping dimensions such as step activity group, execution role, and overhead phase. These labels are analytically useful, but some CI steps serve mixed purposes or are difficult to classify uniquely from telemetry alone. For example, a repository-specific Gradle task may combine setup, orchestration, and execution in a single step. Such ambiguity could affect the precision of the overhead interpretation. We mitigate this threat by using conservative grouping rules, auditing the grouping labels manually in Step 2, and treating the resulting levers as evidence-grounded *first optimization targets* rather than exact causal attributions.

d) Style labeling as dominant operational mechanism.: Execution style is inferred from workflow evidence rather than from explicit self-declared labels. Some repositories mix mechanisms—for example, combining community actions with custom provisioning, or third-party execution with additional local orchestration. This means that style should be understood as the dominant observed operational mechanism, not as a claim that other mechanisms are absent. We mitigate this threat by inheriting the baseline style-detection logic from the taxonomy study [1], preserving evidence fields for auditability, and interpreting cross-style results as comparisons between dominant operational patterns.

e) Usable-verdict and deployment-context constructs.: The updated study design uses *usable verdict* as a practice-facing construct, defined as a run-level terminal outcome of `success` or `failure`. RQ4 also treats trigger event as a deployment-context marker. Both constructs are useful, but neither should be over-interpreted. A usable verdict is still a coarse proxy for operational usefulness, and a trigger event describes deployment context rather than causal intent. We mitigate this threat by defining usable verdict explicitly in the methodology, separating RQ4 from the timing-focused Base regime, and interpreting event patterns as deployment associations rather than causal effects.

B. Internal Validity

a) Confounding from non-random style selection.: Projects do not choose Community, Custom, GMD, or Third-Party styles at random. Style choice may depend on project size, app architecture, test-suite scale, device requirements, team expertise, or organizational constraints. These same factors may also influence timing, predictability, overhead placement, and outcome behavior. We mitigate this threat by using a controlled timing-comparison regime (instrumentation-executed, first-attempt, usable-verdict runs), by separating the broader RQ4 outcome/deployment regime from the timing Base, and by using workflow-shape fingerprints as a downstream robustness layer rather than relying only on pooled comparisons [18, 19].

b) Infrastructure variability and shared-runner effects.: GitHub-hosted CI runs are subject to queueing, transient slowdowns, variable host performance, and external service effects. These sources of variation can inflate tails and sometimes mimic style differences. We mitigate this threat by emphasizing non-parametric comparisons, medians, IQR-based spread, and upper-tail summaries rather than means alone, and by checking whether the main conclusions remain visible under adjacent regimes and workflow-shape robustness settings.

c) Reruns as a distinct operational regime.: Reruns may arise from flaky tests, transient infrastructure issues, or manual intervention, and they can materially change timing behavior. If reruns were pooled with first attempts, they could distort cross-style timing comparisons. We therefore treat reruns as a separate adjacent regime and validate this decision explicitly in Step 3, where rerun usable-verdict records are compared against the main Base subset. This does not eliminate all rerun-related bias, but it reduces the risk of conflating first-attempt performance with recovery behavior [3].

d) Outcome filtering versus representativeness.: The timing-focused Base subset excludes non-usable outcomes, which improves comparability for RQ1–RQ3 but may also remove operationally meaningful failure and cancellation behavior. This creates a tradeoff between cleaner timing comparison and full operational representativeness. We mitigate this threat by making the regime split explicit: RQ1–RQ3 study timing under usable-verdict first attempts, whereas RQ4 studies deployment context and outcome behavior on the broader first-attempt instrumentation-executed regime.

C. External Validity

a) Platform and ecosystem specificity.: Our study focuses on GitHub Actions and Android instrumentation testing, and the results may not generalize directly to other CI platforms or mobile ecosystems. Nevertheless, several observed phenomena—setup/provisioning overhead, heavy-tailed duration behavior, and the value of step-level decomposition—are not unique to Android or to GitHub Actions. We therefore expect parts of the analysis logic to transfer, while specific magnitudes and style mixes may not. Replication on other CI systems and in other mobile stacks remains important future work [34].

b) Repository sample and survivorship bias.: The corpus consists of repositories with detectable instrumentation-testing activity and available historical runs. Repositories with private CI, short retention, sparse activity, or non-standard orchestration may be underrepresented. Likewise, projects that already invest in CI automation may be more visible in the sample than less mature projects. We mitigate this threat by reporting the staged coverage funnel, retaining evidence fields that support replication, and framing the findings as empirical regularities in the observed corpus rather than as universal population estimates.

c) Third-party provider diversity.: The Third-Party style aggregates multiple providers and integration patterns. Provider-specific queueing, allocation policies, and orchestration conventions may dominate the observed behavior for some repositories. Thus, Third-Party results should be interpreted as characterizing the observed provider-backed execution style in the current sample rather than all possible device-farm services. We partially mitigate this threat by treating Third-Party as an operational style-level category and by interpreting provider-sensitive findings with appropriate caution.

d) Sparse support for some styles and signatures.: Custom remains sparse relative to Community, and the exact-signature robustness layer yields only a small number of interpretable within-shape candidates. This limits the generalizability of fine-grained Custom conclusions and of highly constrained within-signature comparisons. We mitigate this threat by treating Custom as a cautious profile, by making the two-tier robustness design explicit, and by distinguishing strongly supported findings from narrower, directionally supported ones.

D. Conclusion Validity

a) Heavy-tailed distributions and subgroup uncertainty.: CI timing outcomes are often heavy-tailed [2], and a small number of extreme runs can distort naive summaries. This risk becomes stronger in small style-specific or within-signature subgroups. We mitigate this threat by using robust summaries, non-parametric tests, and effect sizes, and by interpreting sparse strata conservatively rather than promoting every significant result to a headline conclusion.

b) Multiple comparisons and layered analyses.: The study compares multiple styles across multiple timing outcomes, robustness regimes, and deployment/outcome views. This raises the risk of spurious isolated findings. We mitigate this threat by using Holm correction for pairwise timing comparisons, emphasizing repeated cross-metric patterns over single results, and structuring the synthesis around a small set of operational profile conclusions supported across RQ1–RQ4.

c) Predictability and tail-risk measurement choices.: RQ2 operationalizes predictability using dispersion and tail summaries together with normalized deviation-style measures. These are reasonable for heavy-tailed CI data, but other definitions of predictability are possible. Different choices could change the precise ranking of styles on some measures. We mitigate this threat by combining typical spread, relative tail, and pairwise effect-size evidence rather than relying on a single statistic.

d) From observational profiles to optimization guidance.: The operational profile and decision-support guide are derived from observational evidence, not from controlled interventions. Accordingly, the reported optimization targets are plausible and evidence-grounded, but not experimentally validated causal remedies. Similarly, trigger-conditioned outcome differences in RQ4 should not be interpreted as proof that changing a trigger policy or style will by itself improve success or usable-verdict rates. Future work should validate these inferences through controlled workflow interventions, before/after operational changes, and external replication on matched projects and runner environments.

VIII. Conclusion

Android instrumentation testing is indispensable for validating mobile behavior, yet difficult to operationalize in CI due to configuration/setup overhead, environmental instability, and heavy-tailed performance. Building on an established taxonomy of instrumentation-testing execution styles, this paper presented a telemetry-driven operational characterization of these styles on GitHub Actions. By combining workflow evidence with run/job/step telemetry, we quantified developer-feedback properties across four complementary dimensions: speed, predictability, overhead composition, and run-level verdict usability/deployment context.

To reduce confounding in observational comparisons, we analyzed timing-focused results under a Step 3-validated controlled regime of instrumentation-executed first-attempt runs with usable verdicts, and treated reruns as a distinct adjacent regime. We also introduced a workflow-shape finger- print encoded as a signature hash and used it as a downstream robustness layer to assess whether the main style-level conclusions remain visible across comparable workflow settings.

Overall, the results show that emulator testing styles exhibit distinct operational profiles rather than a single global ranking. Community provides the fastest typical completion but with higher variability and a distributed overhead profile. GMD provides the strongest predictability-first profile, the clearest fast-entry behavior, and an execution-centric overhead structure. Third-Party combines slow sustained execution with the strongest absolute tail-risk profile and a heavy-entry plus heavy-execution overhead pattern. Custom remains a sparse but distinct mixed profile, with competitive entry, weaker predictability, and a large completion tail. RQ4 further shows that styles also differ in usable-verdict rate, success rate among usable outcomes, and deployment context, indicating that style choice should be evaluated not only as a timing decision but also as a trigger- and outcome-sensitive oper- ational decision.

Taken together, these findings suggest that style selection and optimization should be objective-driven and phase-aware. Improvements are most reliable when they target the dominant source of latency or instability for the relevant workflow regime, rather than treating instrumentation testing as a single undifferentiated CI task. To support this, we synthesized the RQ findings into an interpretable operational profile and a profile-derived decision-support guide that maps practitioner objectives to style choices and first optimization targets.

Future work: Future work includes (i) prospectively validating the profile- derived optimization targets and decision-support guidance via controlled workflow interventions, including trigger-aware before/after studies and matched comparisons across feasible style choices; (ii) extending workflow-shape fingerprints with richer structural, environment, and provider-sensitive signals to better capture execution-shape

confounding in observational CI telemetry; and (iii) generalizing the approach to other CI platforms and mobile ecosystems to test the portability of the style profiles, outcome patterns, and decision-support guid- ance.

IX. Appendix

A. Detailed Validation Rules and Additional Results

This appendix provides the supporting detail for the validation procedure summarized in Section III-G. In the main paper, Step 1 reports a compact category-level summary of the automated rule-based validation, while Step 3 reports the high-level outcome of the robustness-check analysis. Here, we provide the corresponding appendix-level detail: the full Step 1 rule set, the explicit timing-equation checks, the interpretation of telemetry-dependent rules, the detailed rule-level results in Table IV, and the supporting summary for Step 3 in Table V.

1) Step 1 purpose and scope: The goal of Step 1 is to establish technical data-quality validity before the RQ-specific analyses begin. In particular, the automated checks verify that: (i) the constructed run$\times$style records are uniquely and correctly keyed, (ii) required identifiers and study fields are present, (iii) run boundaries and selected cutpoints obey the expected temporal ordering, (iv) the Layer 1 and Layer 2 timing decompositions are internally coherent, (v) the selected invocation and execution-end cutpoints can be traced back to unique step-level source records, and (vi) the derived support fields used for comparability and robustness are present and internally consistent.

As in the main paper, we report the automated checks for both the full dataset and for Base, the study's main controlled timing regime defined by instrumentation-executed first-attempt usable-verdict records. The main text summarizes these results at the category level using average applicable-record accuracy across rules. This appendix provides the rule-level detail behind that summary.

2) Detailed rule set: The automated validation comprised 53 rules:

1) **V1 Key uniqueness**: verifies that each record is uniquely identified by the study key combination.
2) **V2 Required identifiers present**: checks the presence of required identifiers such as workflow identifier and workflow ID.
3) **V3 Run-bounds order**: checks that run start time is less than or equal to run end time.
4) **V4 Window-bounds order**: checks that execution-window start time is less than or equal to execution-window end time.
5) **V5 Cutpoint temporal order**: checks that run start $\leq$ invocation start $\leq$ execution end $\leq$ run end.
6) **V6 Window inside run**: checks that the execution window lies within the overall run boundaries.
7) **V7 Run-duration non-negativity**: checks that total run duration is non-negative.
8) **V8 Layer 2 non-negativity**: checks that the three Layer 2 components are non-negative.
9) **V9 Components bounded by run duration**: checks that each Layer 2 component does not exceed total run duration.
10) **V10 Layer 1 sum-to-run equation**: checks that the sum of the three Layer 1 components equals run duration.
11) **V11 Layer 2 sum-to-run equation**: checks that the sum of the three Layer 2 components equals run duration.
12) **V12 Pre-invocation recomputation**: checks that the stored pre-invocation duration matches recomputation from timestamps.
13) **V13 Invocation-window recomputation**: checks that the stored invocation execution window matches recomputation from timestamps.
14) **V14 Post-invocation recomputation**: checks that the stored post-invocation duration matches recomputation from timestamps.
15) **V15 Unique invocation cutpoint selection**: checks that exactly one invocation cutpoint is selected from the step-level source data.
16) **V16 Unique execution-end cutpoint selection**: checks that exactly one execution-end cutpoint is selected from the step-level source data.
17) **V17 Invocation-step source match**: checks that the stored invocation-step fields match the selected source step record.
18) **V18 Execution-end-step source match**: checks that the stored execution-end-step fields match the selected source step record.
19) **V19 Style-scope validity**: checks that each record belongs to one of the study styles included in scope.
20) **V20 Base-flag presence**: checks that the study's base-condition flag is present.
21) **V21 Robust-flag presence**: checks that the study's robustness-condition flag is present.
22) **V22 Style-level distinct-job-count presence**: checks that the style-level distinct-job-count field is present.
23) **V23 Style-level distinct-job-base-name-count presence**: checks that the style-level distinct-job-base-name-count field is present.
24) **V24 Style-level matrix-like-job-count presence**: checks that the style-level matrix-like-job-count field is present.
25) **V25 Style-level matrix-expansion-flag presence**: checks that the style-level matrix-expansion flag is present.
26) **V26 Style-level parallel-same-style-flag presence**: checks that the style-level parallel-same-style flag is present.
27) **V27 Style-level max-parallel-jobs presence**: checks that the style-level max-parallel-jobs field is present.
28) **V28 Style-level repeated-same-style-flag presence**: checks that the style-level repeated-same-style flag is present.
29) **V29 Style-level invocation-candidate-count presence**: checks that the style-level invocation-candidate count is present.

30) **V30 Style-level distinct-invocation-step-name-count presence**: checks that the style-level distinct invocation-step-name count is present.
31) **V31 Style-level invocation-candidate-step-name-list presence**: checks that the style-level invocation-candidate step-name list is present.
32) **V32 Style-level complexity-class presence**: checks that the style-level complexity class is present.
33) **V33 Invocation-candidate-total-count presence**: checks that the invocation-candidate total count is present.
34) **V34 Invocation-candidate anchor-match-count presence**: checks that the anchor-match candidate count is present.
35) **V35 Invocation-candidate explicit-instrumentation-count presence**: checks that the explicit-instrumentation candidate count is present.
36) **V36 Invocation-candidate custom-supported-count presence**: checks that the custom-supported candidate count is present.
37) **V37 Distinct invocation-candidate step-name-count presence**: checks that the distinct invocation-candidate step-name count is present.
38) **V38 Distinct invocation-candidate job-count presence**: checks that the distinct invocation-candidate job count is present.
39) **V39 Selected invocation-priority-source presence**: checks that the selected invocation-priority source is present.
40) **V40 Execution-window candidate-count presence**: checks that the execution-window candidate count is present.
41) **V41 Execution-window distinct-job-count presence**: checks that the execution-window distinct-job count is present.
42) **V42 Cross-job execution-window-flag presence**: checks that the cross-job execution-window flag is present.
43) **V43 Job-base-name count bounded by distinct-job count**: checks that the job-base-name count does not exceed the distinct-job count.
44) **V44 Matrix-like-job count bounded by distinct-job count**: checks that the matrix-like-job count does not exceed the distinct-job count.
45) **V45 Max-parallel-jobs bounded by distinct-job count**: checks that the max-parallel-jobs value does not exceed the distinct-job count.
46) **V46 Distinct invocation-step-name proxy bounded by total invocation-candidate proxy count**: checks that the distinct invocation-step-name proxy count does not exceed the total invocation-candidate proxy count.
47) **V47 Invocation-candidate partition sums to total**: checks that the candidate subcounts sum to the total invocation-candidate count.
48) **V48 Execution-window distinct-job count bounded by total candidate count**: checks that the distinct-job count within the execution window does not exceed the total candidate count.
49) **V49 Matrix-expanded style implies repeated same-style structure**: checks logical consistency between matrix-expanded and repeated-same-style indicators.
50) **V50 Cross-job execution-window flag consistency**: checks that the cross-job execution-window flag agrees with the corresponding distinct-job count.
51) **V51 Parallel same-style flag consistency**: checks that the parallel-same-style flag agrees with the max-parallel-jobs field.
52) **V52 Selected invocation-priority-source validity and contextual coherence**: checks that the selected invocation-priority source takes a valid current pipeline value and is coherent with the selected invocation state and corresponding candidate-support fields.
53) **V53 Style-level complexity-class presence**: checks that the style-level complexity class is present.

3) Validation of Layer 1 and Layer 2 timing equations: The automated procedure explicitly validates the study's two-layer timing model. As described in the main paper, Layer 1 and Layer 2 both decompose the same run duration at different levels of resolution.

For **Layer 1**, validity is checked directly through Rule V10:

$$\begin{aligned}\text{Run Duration} = {} & \text{Time to Instrumentation Envelope} \\ & + \text{Instrumentation Job Envelope} \\ & + \text{Post-Instrumentation Tail.}\end{aligned} \tag{1}$$

For **Layer 2**, validity is checked through timestamp-based recomputation from the selected cutpoints and run boundaries:

$$\text{Pre-Invocation} = \text{Invocation Start} - \text{Run Start.} \tag{2}$$

$$\begin{aligned}\text{Invocation Execution Window} = {} & \text{Execution End} \\ & - \text{Invocation Start.}\end{aligned} \tag{3}$$

$$\text{Post-Invocation} = \text{Run End} - \text{Execution End.} \tag{4}$$

Rules V12–V14 compare these recomputed values to the stored Layer 2 durations, while Rules V5 and V6 verify the temporal admissibility of the underlying cutpoints. Together, these checks confirm that the Layer 2 decomposition is both temporally and arithmetically consistent with the recorded run boundaries and selected step-level anchors.

4) Interpretation of missing and non-applicable cases: Several Step 1 rules depend on the availability of step-level telemetry. Accordingly, a record may be *missing* for a given rule not because the dataset is contradictory, but because the relevant boundary is not directly observable in that run. This is especially relevant for Layer 2 and source-traceability checks, which are defined only when the necessary cutpoints are available from step telemetry.

For that reason, the main-paper summary reports *applicable-record accuracy*, computed as:

$$\frac{\texttt{ok}}{\texttt{ok} + \texttt{mismatch}},$$

excluding missing or non-applicable cases. This interpretation is consistent with the Step 1 summary in the main paper, where telemetry-dependent categories remain perfect or near-perfect once evaluated on applicable records rather than on all rows.

5) Detailed Step 1 results: Table IV reports the full rule-level results underlying the category summary shown in the main paper. The table includes both the full dataset and Base.

6) Appendix-level takeaway: Taken together, the detailed Step 1 results confirm the interpretation given in the main paper: the automated validation establishes strong technical internal consistency for the constructed dataset. Structural rules, Layer 1 equation checks, Layer 2 recomputation checks, and auxiliary consistency are fully satisfied on applicable records. The only residual non-perfect cases are rare and limited to temporal/window-placement checks outside Base and a small number of source-traceability mismatches. This appendix therefore provides the detailed audit basis for the main paper's claim that the downstream timing analyses rest on an internally coherent measurement foundation.

7) Step 3 robustness-check support: Table V summarizes the supporting evidence for Validation Step 3. The controller-regime check compares the median-based style ordering in adjacent run×style regimes against the Base reference ordering. The workflow-shape check summarizes the strongest exact-signature candidates available for deeper within-shape follow-up. Together, these results support the use of Base as the study's main comparison regime while showing that the exact-signature layer is informative but only partially sufficient for balanced four-style robustness confirmation.

REFERENCES

[1] Hamid Parsazadeh, Taher A. Ghaleb, and Safwat Hassan. Android instrumentation testing in continuous integration: Practices, patterns, and performance. 2026.

[2] T.A. Ghaleb et al. An empirical study of the long duration of continuous integration builds. *Empirical Software Engineering*, 2019.

[3] J. Huang et al. On the reruns of github actions workflows. *Proceedings of the ACM on Software Engineering*, 2024.

[4] Nimmi Rashinika Weeraddana, Mahmoud Alfadel, and Shane McIntosh. Characterizing timeout builds in continuous integration. *IEEE Transactions on Software Engineering*, 2024.

[5] Tim Kinsman, Mairieli Wessel, Marco A. Gerosa, and Christoph Treude. How do software developers use github actions to automate their workflows? In *Proceedings of the 18th International Conference on Mining Software Repositories (MSR)*, 2021.

[6] Alexandre Decan, Tom Mens, and Hassan Onsori Delicheh. On the outdatedness of workflows in the github actions ecosystem. *Journal of Systems and Software*, 206:111827, 2023.

[7] Android Developers. Build instrumented tests. https://developer.android.com/training/testing/instrumented-tests, 2025. Accessed 2026-03-04.

[8] Android Developers. Set up your project for androidx test. https://developer.android.com/training/testing/instrumented-tests/androidx-test-libraries/test-setup, 2025. Accessed 2026-03-04.

[9] Android Developers. Espresso. https://developer.android.com/training/testing/espresso, 2025. Accessed 2026-03-04.

[10] A. Tahir et al. Test flakiness' causes, detection, impact and responses. *Journal of Systems and Software*, 2023.

[11] V. Pontillo et al. Test code flakiness in mobile apps: The developer's perspective. *Information and Software Technology*, 2024.

[12] Antonio Romano, Weihang Wang, Andy Zaidman, Maurício Aniche, and Sebastiano Panichella. An empirical analysis of ui-based flaky tests. In *Proceedings of the International Conference on Software Engineering (ICSE)*, 2021.

[13] Steve T. C. S. N. Meng, Nachiappan Nagappan, et al. An empirical study of flaky tests in android apps. In *Proceedings of the International Conference on Software Engineering (ICSE)*, 2019.

[14] Firebase. Get started with instrumentation tests (firebase test lab). https://firebase.google.com/docs/test-lab/android/instrumentation-test, 2026. Accessed 2026-03-04.

[15] Firebase. Start testing with the gcloud cli (firebase test lab). https://firebase.google.com/docs/test-lab/android/command-line, 2026. Accessed 2026-03-04.

[16] GitHub Docs. Rest api endpoints for workflow runs. https://docs.github.com/rest/actions/workflow-runs, 2026. Accessed 2026-02-12.

[17] GitHub Docs. Rest api endpoints for workflow jobs. https://docs.github.com/en/rest/actions/workflow-jobs, 2026. Accessed 2026-02-12.

[18] Paul R. Rosenbaum and Donald B. Rubin. The central role of the propensity score in observational studies for causal effects. *Biometrika*, 70(1):41–55, 1983.

[19] Miguel A. Hernán and James M. Robins. *Causal Inference: What If*. Chapman and Hall/CRC, 2024. Online version.

[20] Pablo Valenzuela-Toledo and Alexandre Bergel. Evolution of github action workflows. In *Proceedings of the International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2022.

[21] Pablo Valenzuela-Toledo, Alexandre Bergel, and Timo Kehrer. The hidden costs of automation: An empirical study on github actions workflow maintenance. https://arxiv.org/abs/2409.02366, 2024. Accessed 2026-03-04.

[22] Anonymous et al. An empirical study of the evolution of github actions workflows. arXiv preprint, 2026. arXiv:2602.14572.

[23] Nan Yang, Pieter J. L. Cuijpers, Ramon R. H. Schiffelers, Johan Lukkien, and Alexander Serebrenik. An interview study of how developers use execution logs in embedded software engineering. In *Proceedings of the 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 61–70, 2021.

[24] Maria Carla Calzarossa and Giuseppe Serazzi. Workload characterization: A survey. *Proceedings of the IEEE*, 88(11):1712–1733, 2000.

[25] Christina Delimitrou and Christos Kozyrakis. Quasar: Resource-efficient and qos-aware cluster management. In *Proceedings of the 19th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2014.

[26] NIST. Fips pub 180-4: Secure hash standard (shs). https://csrc.nist.gov/publications/detail/fips/180/4/final, 2015. Accessed 2026-02-25.

[27] Thomas Rausch et al. An empirical analysis of build failures in the continuous integration workflows of java-based open-source software projects. In *Proceedings of the Mining Software Repositories (MSR)*, 2017.

[28] Carmine Vassallo et al. A tale of ci build failures: An open source and a financial organization perspective. In *Proceedings of the International Conference on Software Maintenance and Evolution (ICSME)*, 2017.

[29] Leo Breiman, Jerome H. Friedman, Richard A. Olshen, and Charles J. Stone. *Classification and Regression Trees*. Chapman and Hall/CRC, New York, 1984.

[30] Cynthia Rudin. Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1(5):206–215, 2019.

[31] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. *Experimentation in Software Engineering*. Springer, 2012.

[32] Per Runeson and Martin Höst. Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2):131–164, 2009.

[33] GitHub Docs. Managing github actions settings for a repository (artifact and log retention). https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/enabling-features-for-your-repository/managing-github-actions-settings-for-a-repository, 2026. Accessed: 2026-02-24.

[34] Francesco Moriconi et al. Ghalogs: Large-scale dataset of github actions runs. In *Proceedings of the Mining Software Repositories (MSR)*, 2025. Dataset paper (PDF available online).

TABLE IV

DETAILED STEP 1 AUTOMATED VALIDATION RESULTS BY RULE. OVERALL AND BASE VALUES REPORT APPLICABLE-RECORD ACCURACY, COMPUTED AS `OK / (OK + MISMATCH)`. RULES WITH NO APPLICABLE RECORDS IN A GIVEN VIEW ARE REPORTED AS `N/A`.

Rule	Description	Overall	Base
V1	Key uniqueness	100.00%	100.00%
V2	Required identifiers present	100.00%	100.00%
V3	Run bounds ordered	100.00%	100.00%
V4	Execution-window bounds ordered	100.00%	100.00%
V5	Cutpoints in temporal order	99.10%	100.00%
V6	Execution window lies within run bounds	99.10%	100.00%
V7	Run duration non-negative	100.00%	100.00%
V8	Layer 2 durations non-negative	100.00%	100.00%
V9	Layer 2 components bounded by run duration	100.00%	100.00%
V10	Layer 1 sum equals run duration	100.00%	100.00%
V11	Layer 2 sum equals run duration	100.00%	100.00%
V12	Pre-invocation recomputation	100.00%	100.00%
V13	Invocation-window recomputation	100.00%	100.00%
V14	Post-invocation recomputation	100.00%	100.00%
V15	Exactly one invocation cutpoint selected	99.86%	99.86%
V16	Exactly one execution-end cutpoint selected	99.86%	99.86%
V17	Invocation-step fields match source step record	99.88%	99.88%
V18	Execution-end-step fields match source step record	100.00%	100.00%
V19	Style is within study scope	100.00%	100.00%
V20	Base flag present	100.00%	100.00%
V21	Robust flag present	100.00%	100.00%
V22	Style-level distinct job count present	100.00%	100.00%
V23	Style-level distinct job base-name count present	100.00%	100.00%
V24	Style-level matrix-like job count present	100.00%	100.00%
V25	Style-level matrix-expansion flag present	100.00%	100.00%
V26	Style-level parallel same-style flag present	100.00%	100.00%
V27	Style-level max-parallel-jobs field present	100.00%	100.00%
V28	Style-level repeated same-style flag present	100.00%	100.00%
V29	Style-level invocation-candidate count present	100.00%	100.00%
V30	Style-level distinct invocation-step-name count present	100.00%	100.00%
V31	Style-level invocation-candidate step-name list consistency	100.00%	100.00%
V32	Style-level complexity class present	100.00%	100.00%
V33	Invocation-candidate count total present	100.00%	100.00%
V34	Stage-1 anchor candidate count present	100.00%	100.00%
V35	Explicit-instrumentation candidate count present	100.00%	100.00%
V36	Custom-supported candidate count present	100.00%	100.00%
V37	Distinct invocation-candidate step-name count present	100.00%	100.00%
V38	Distinct invocation-candidate job count present	100.00%	100.00%
V39	Selected invocation-priority source present	100.00%	100.00%
V40	Execution-window candidate count present	100.00%	100.00%
V41	Execution-window distinct-job count present	100.00%	100.00%
V42	Cross-job execution-window flag present	100.00%	100.00%
V43	Style job-base count $\leq$ distinct job count	100.00%	100.00%
V44	Matrix-like count $\leq$ distinct job count	100.00%	100.00%
V45	Max parallel jobs $\leq$ distinct job count	100.00%	100.00%
V46	Distinct invocation-step-name proxy $\leq$ invocation-candidate total	100.00%	100.00%
V47	Invocation-candidate partition consistency	100.00%	100.00%
V48	Execution-window distinct-job count $\leq$ execution-window candidate count	100.00%	100.00%
V49	Matrix-expanded implies repeated-style consistency	100.00%	100.00%
V50	Cross-job execution-window flag consistency	100.00%	100.00%
V51	Parallel same-style flag consistency	100.00%	100.00%
V52	Selected invocation-priority-source validity and contextual coherence	100.00%	100.00%
V53	Style-level complexity-class presence	100.00%	100.00%

TABLE V
SUPPORT FOR VALIDATION STEP 3 ROBUSTNESS CHECKS.

Check	Target	Finding	Interpretation
Controller-regime	`all_run_per_style` vs. Base	Broad style ordering is consistent with the Base reference.	Supports Base as a stable main comparison regime.
Controller-regime	`attempt_eq_1` vs. Base	Style ordering remains consistent with the Base reference.	Suggests Base is not an arbitrary narrowing of first-attempt runs.
Controller-regime	`rerun_usable_verdict` vs. Base	Rerun usable-verdict records behave as a distinct adjacent regime.	Supports analyzing reruns separately; the distinctness of this adjacent regime is also confirmed by the focused statistical comparison reported in Validation Step 3.
Workflow-shape	Exact-signature candidates	Only two exact signatures provide interpretable cross-style overlap, and neither yields a balanced four-style setting.	Exact-signature robustness is informative but only partially confirmatory.
Workflow-shape	Two-tier follow-up	Exact signatures are retained as a strict first-tier lens, complemented by a coarsened shape-family analysis for broader cross-style coverage.	Provides a practical basis for later within-shape conclusion checks.